

PPCWay

Google Ads Onboarding & Auto-Optimization Platform

Technical Architecture & Implementation Blueprint

Version 1.0

Prepared for	A1 Inc. — PPCWay.com
Responds to	PPCWay Developer Requirements Document (DRD) v1.0
Document type	Technical response, architecture & delivery plan
Date	13 August 2026
Classification	Confidential — Internal & Vendor/Developer Use Only

Purpose of this document. The DRD defines *what* PPCWay must do. This document defines *how* it will be built: the system architecture, the technology stack and the reasoning behind it, every external account and credential required to operate the platform, the LLM selection and cost model, the hard platform limitations that constrain the product, and a phased delivery plan with effort estimates. Section 19 answers each of the open questions raised in DRD §13.

Reading paths. Product owners: §1, §2, §11, §15, §16, §17, §19. Engineers: §3–§10, §13, §18, appendices. Finance/procurement: §11, §17. Compliance/legal: §12, §13.

Contents

1	Executive Summary	5
1.1	The five decisions that define this build	5
1.2	What the architecture looks like in one paragraph	5
1.3	What will constrain the product, and must be accepted up front	5
1.4	Delivery summary	6
2	Scope, Assumptions and How to Use This Document	6
2.1	In scope for this document	6
2.2	Stated assumptions	6
2.3	Terminology	7
3	Technology Stack	7
3.1	Backend language and framework: Python 3.12 + FastAPI	7
3.2	Full recommended stack	8
3.3	Why a durable workflow engine, not a job queue	8
3.4	Runtime topology	9
3.5	Cloud provider	9
4	System Architecture	10
4.1	Architecture principles	10
4.2	High-level architecture	11
4.3	Component responsibilities	11
4.4	Flow 1: Onboarding to first live campaign	12
4.5	Flow 2: The optimization loop	13
4.6	Multi-tenancy and isolation	13
4.7	Frontend design constraints	14
5	Google Ads API Integration Layer	14
5.1	Access tiers: what each one permits	14
5.2	Account structure	15
5.3	OAuth and token lifecycle	15
5.4	Quota model: sizing the platform against the operation ceiling	16
5.5	The guardrail layer	18
5.6	Idempotency, retries and backoff	18
5.7	Detecting disapprovals, suspensions and billing holds	19
5.8	API version upgrade process	19
5.9	Services used, mapped to features	20
6	Campaign Generation Engine	20
6.1	Pipeline	20
6.2	Website crawl	21
6.3	Keyword theme generation	21
6.4	Structure decision rules engine	22
6.5	Bidding strategy decision table	22
6.6	Negative keyword strategy	23
6.7	Ad copy generation and creative constraints	23
7	LLM Strategy and Selection	24
7.1	Where the LLM is used, and where it must not be	24
7.2	Model selection	25
7.3	Prompt architecture and caching	25
7.4	The deterministic policy validator	26
7.5	LLM failure modes and mitigations	27
7.6	Provider abstraction	27

7.7	LLM cost model	27
8	Optimization Engine	28
8.1	Job catalogue	28
8.2	Rules, not models	30
8.3	Autonomy levels	30
8.4	The Optimization Action Log	31
8.5	Anomaly detection approach	31
9	Data Model	32
9.1	Core relational schema	33
9.2	Metrics store	34
9.3	Data retention and deletion	35
10	Reporting, Notifications and Internal Admin	35
10.1	Merchant dashboard	35
10.2	Weekly digest email	36
10.3	Notifications	36
10.4	Internal admin console	36
11	Required Accounts, Credentials and API Keys	37
11.1	Google (largest and slowest set)	37
11.2	Anthropic	38
11.3	Payments and email	38
11.4	Infrastructure and tooling	39
11.5	Secret management design	39
11.6	Environment variable reference	40
12	Security and Compliance	40
12.1	Credential and token protection	40
12.2	Authentication and authorization	41
12.3	Google Ads API policy compliance and RMF	41
12.4	Data privacy: PIPEDA and GDPR	42
12.5	PCI scope	43
12.6	Crawler isolation and SSRF defence	43
12.7	Dependency and supply-chain hygiene	43
13	Limitations and Constraints	44
13.1	The billing hand-off cannot be eliminated	44
13.2	Account provisioning has a hard weekly ceiling	44
13.3	API quota ceilings bound merchant count	45
13.4	The API version treadmill is permanent	45
13.5	Conversion tracking is the single point of failure	45
13.6	Google Ads platform entity limits	45
13.7	What the Optimization Engine cannot do	46
13.8	LLM limitations	46
13.9	Commercial and policy limitations	47
13.10	Explicitly out of scope for v1	47
14	Risk Register	47
15	Non-Functional Requirements and Service Objectives	48
15.1	Observability	49
16	Delivery Plan and Effort Estimates	49
16.1	Phase 0 — Foundations and credentials (3–4 weeks elapsed)	49
16.2	Phase 1 — MVP (14–18 weeks elapsed)	49
16.3	Phase 2 — Automation depth (10–14 weeks)	50

16.4 Phase 3 — Scale and agency (12–16 weeks)	50
16.5 Critical path and sequencing	51
16.6 Team shape	51
17 Running Cost Model	51
18 Environments, CI/CD and Testing	52
18.1 Environments	52
18.2 CI/CD	52
18.3 Testing strategy	53
19 Answers to the DRD's Open Questions	53
19.1 Q1: Confirm target Google Ads API access tier and current quota limits	53
19.2 Q2: Confirm whether PPCWay operates a single MCC or a hierarchy of sub-MCCs	54
19.3 Q3: Confirm the LLM provider and the content-safety / policy-compliance validation approach	54
19.4 Q4: Confirm initial supported business verticals for launch	55
19.5 Q5: Confirm hosting/cloud provider preference and any existing A1 Inc. infrastructure to reuse	55
A Appendix A — Glossary	56
B Appendix B — Reference GAQL Queries	57
C Appendix C — Configuration Reference	58
D Appendix D — Phase Acceptance Criteria	59
D.1 Phase 0	59
D.2 Phase 1 (MVP)	59
D.3 Phase 2	59
D.4 Phase 3	59

1 Executive Summary

PPCWay is a multi-tenant B2B SaaS platform that automates the full lifecycle of Google Ads management for small and medium businesses: account connection, guided onboarding, campaign generation, launch, and continuous optimization. This document specifies the build.

1.1 The five decisions that define this build

Everything else follows from these five choices. Each is justified in the section indicated.

#	Decision	Choice	Section
1	Backend language	Python 3.12 (FastAPI), because Google publishes an <i>official</i> , same-day-updated Google Ads API client for Python and none for Node.js	§3.1
2	Workflow engine	Temporal.io for the Optimization Engine — durable execution, replayable history, and per-step retry are effectively mandatory given the auditability and reversibility requirements of DRD §5.5	§3.3
3	Google Ads access tier	Standard Access under a single PPCWay MCC, with Basic Access as the interim tier for closed beta only	§5.1
4	LLM	Claude Opus 5 (claude-opus-5) via the Anthropic API as the default for every generative step; every output passes a deterministic policy validator before it can reach Google. §7.7 gives the cost-down options and the trade-off, so A1 Inc. can choose	§7
5	Autonomy model	Every automated mutation flows through a single guardrail layer that is the <i>only</i> code path permitted to call a Google Ads <code>Mutate</code> method	§5.5, §8.3

1.2 What the architecture looks like in one paragraph

A Next.js frontend on `ppcway.com` talks to a FastAPI application tier. Onboarding writes a Business Profile; that triggers a Temporal workflow that crawls the merchant's site, generates keyword themes and ad copy (LLM), assembles a campaign structure (deterministic rules engine), runs a policy validator, and renders a plain-English preview. On merchant approval, a single Google Ads Gateway service — the only component holding Google credentials — executes the mutations. From then on, scheduled and event-driven Temporal workflows read performance data via GAQL into a metrics store, evaluate optimization rules, and apply changes through the same gateway, writing to an append-only Optimization Action Log before and after every mutation.

1.3 What will constrain the product, and must be accepted up front

These are not implementation risks that better engineering removes. They are properties of the platforms PPCWay is built on. Full detail in §13.

Constraint	Consequence for PPCWay
Google will not accept payment-card data through the API; a payments profile can only be created in the Google Ads UI	Onboarding cannot be fully self-contained. Every merchant must complete a hand-off to Google's own billing pages before any campaign can serve. This is a permanent step in the funnel, not a v1 shortcut.
New-account creation is capped at 15 accounts per week on an automatic or manual payments profile (1,500/week only with monthly invoicing)	The Phase 2 "PPCWay creates your Google Ads account for you" flow has a hard throughput ceiling until A1 Inc. qualifies for monthly invoicing with Google.
Basic Access allows 15,000 API operations/day across the entire developer token	At a realistic ~40 ops/merchant/day, Basic Access supports roughly 300–375 merchants <i>in total</i> , not per account. Standard Access is a launch blocker, not an optimization.
Google Ads API ships a new major version monthly with a 12-month support window	A recurring, non-negotiable maintenance obligation of roughly 2–4 engineer-days per quarter, forever.
Standard Access requires Required Minimum Functionality (RMF) compliance	PPCWay's deliberately simplified UI must still expose a specific set of creation, management and reporting capabilities. This shapes the v1 UI scope. See §12.3.

Constraint	Consequence for PPCWay
Conversion tracking is the single point of failure for the entire optimization thesis	Without working conversion data the Optimization Engine cannot compute CPA or ROAS and degrades to a click optimizer. Onboarding must treat conversion-tag verification as a blocking gate, not a nice-to-have.

1.4 Delivery summary

Phase	Scope	Team	Elapsed
Phase 0	Credentials, MCC, developer token application, environments, CI/CD	1 backend + 1 DevOps	3–4 weeks (gated by Google review)
Phase 1 (MVP)	Auth, OAuth linking, onboarding wizard, Search campaign generation, Approve-First optimization, basic dashboard, Stripe	2 backend + 1 frontend + 0.5 DevOps	14–18 weeks
Phase 2	Account provisioning, Performance Max, Full-Auto mode, weekly digest, expanded reporting	2 backend + 1 frontend	10–14 weeks
Phase 3	Agency console, white-label, advanced anomaly detection	2 backend + 1.5 frontend	12–16 weeks

Phase 1 cannot be compressed below roughly 14 weeks without cutting scope, because the Google Ads API integration and the guardrail/audit layer are on the critical path and cannot be parallelised past a certain point. Detail and assumptions in §16.

2 Scope, Assumptions and How to Use This Document

2.1 In scope for this document

The functional requirements in DRD §5 are taken as given and are not re-litigated here. What this document adds is: architecture, technology selection with rationale, credential and account inventory, LLM selection and cost modelling, the complete data model, security and compliance design, the limitation register, risk register, delivery plan, and running-cost model.

2.2 Stated assumptions

Where the DRD leaves a decision open, this document takes a position and labels it. Each assumption below is revisited in §19 with the question it answers.

#	Assumption	Basis	If wrong
A1	A1 Inc. will apply for Google Ads API Standard Access before public launch, and will complete Google Cloud brand verification to expedite review	DRD §6.4 states Standard Access is required	Launch must be capped at a few hundred merchants; §5.1
A2	PPCWay operates a single Manager Account (MCC) in Phase 1–2, with sub-MCC sharding deferred to Phase 3	Simplest structure that satisfies the requirement; 85,000-account link ceiling is far beyond Phase 1–2 volume	Re-shard later; the design keeps <code>mcc_id</code> on every link record to make this non-breaking
A3	Merchants pay Google directly. PPCWay never takes custody of ad spend	DRD §5.7, §12	Money-transmission and PCI scope change materially; out of scope for this document
A4	Launch verticals exclude Google's restricted categories (healthcare, financial services, gambling, adult, political)	DRD §5.3 requires blocking these	Restricted-category certification logic must be built in Phase 1, adding ~3 weeks

#	Assumption	Basis	If wrong
A5	Anthropic's Claude API is the LLM provider	DRD §7 names it	§7.2 gives the substitution criteria; the abstraction boundary is defined in §7.6
A6	Hosting is Google Cloud Platform	DRD §6.8 notes natural affinity; A1's existing "A2 stack" may override	§3.5 lists the AWS-equivalent services; the design is not GCP-locked
A7	English-language, Canada + US markets at launch	A1 Inc. is Ontario-based	Multi-locale ad copy and locale-specific policy rules add ~4 weeks

2.3 Terminology

Google Ads terms are used with their exact API meanings. Appendix A is a glossary. Two distinctions matter throughout:

- **Operation vs request.** Google Ads API quota is consumed per *operation*, and a single request can carry many operations (up to 10,000 mutate operations). Batching reduces request count but **not** operation count. This distinction drives the entire quota model in §5.4.
- **Mutate vs read.** Both consume quota. Read-heavy reporting is therefore not free, which is why §9.2 mandates a pre-aggregated metrics store rather than live GAQL on page load.

3 Technology Stack

3.1 Backend language and framework: Python 3.12 + FastAPI

Recommendation: Python. The DRD leaves this open between Node.js and Python. The deciding factor is not language preference; it is the Google Ads API client library situation.

Google publishes and maintains official Google Ads API client libraries for **Java**, **.NET**, **PHP**, **Python**, **Ruby** and **Perl**. There is **no official Node.js library** — Node.js and Go are served only by community-maintained clients. Since January 2026 the Google Ads API ships a new major version monthly with a 12-month support window per version, which means a permanent migration treadmill. On the official Python client, support for a new API version lands on the day of release. On a community Node.js client, PPCWay's ability to migrate depends on a third-party maintainer's schedule.

For a product whose entire value proposition is a Google Ads integration, taking a dependency on a community client for the most critical vendor SDK is the wrong risk to accept. The rest of the Python case is corroborating rather than decisive:

Factor	Python	Node.js/TypeScript
Google Ads API client	Official (google-ads), same-day version support	Community only (e.g. Opteo's google-ads-api)
Temporal SDK maturity	First-class	First-class
Anthropic SDK	First-class (anthropic)	First-class (@anthropic-ai/sdk)
Crawling / HTML parsing	Very strong (httpx, selectolax, Playwright)	Strong (playwright, cheerio)
Numeric / statistical work for anomaly detection	Native strength (numpy, pandas, scipy)	Weaker; usually needs a service boundary
Shared types with the frontend	Requires codegen (OpenAPI to TypeScript)	Shared types natively
Single-language full stack	No	Yes

The honest trade-off: choosing Python means the team maintains two languages (Python backend, TypeScript frontend) and generates frontend types from the OpenAPI schema rather than sharing them directly.

That is a real cost, and it is smaller than the cost of being unable to migrate the Google Ads integration on Google's schedule.

If A1 Inc. mandates Node.js — for example because the existing “A2 stack” is TypeScript and consistency outweighs the above — the architecture in this document is unchanged apart from §5. In that case: use NestJS, use Opteo's `google-ads-api`, budget an additional 1–2 engineer-weeks per major API version for migration lag, and add a contingency plan for the case where the community client lags a breaking Google change (an internal gRPC/REST fallback for affected services).

3.2 Full recommended stack

Layer	Choice	Notes
Frontend	Next.js 15 (App Router), TypeScript, Tailwind CSS	SSR for marketing pages (SEO), client-side for the app shell
UI components	shadcn/ui or Radix primitives	Avoid data-grid-heavy “ads platform” defaults; see §4.7
Backend API	Python 3.12, FastAPI, Pydantic v2	OpenAPI generated automatically, feeds frontend type generation
API style	REST + JSON	GraphQL adds schema complexity with no benefit at this scope
Auth	Managed identity provider (Auth0 / Clerk / Firebase Auth)	§12.2 — do not hand-roll password and MFA handling
Workflow / jobs	Temporal.io (Temporal Cloud)	§3.3
Lightweight queue / cache	Redis (Cloud Memorystore)	Rate-limit token buckets, dashboard cache, idempotency keys
Primary database	PostgreSQL 16 (Cloud SQL)	§9.1
Metrics store	PostgreSQL + TimescaleDB in Phase 1–2; BigQuery from Phase 3	§9.2
Object storage	Google Cloud Storage	Crawled assets, generated images, exported reports
Secrets	Google Secret Manager + Cloud KMS envelope encryption	§12.1
Google Ads client	<code>google-ads</code> (official Python)	§5
LLM client	<code>anthropic</code> (official Python SDK)	§7
Billing	Stripe (Billing + Checkout + Customer Portal)	§11
Transactional email	Postmark (or A1's existing provider)	§11
Containers	Docker, deployed to Cloud Run	§3.4
CI/CD	GitHub Actions	§18.2
Observability	OpenTelemetry to Google Cloud Operations; Sentry for errors	§15.1
IaC	Terraform	§18.2

3.3 Why a durable workflow engine, not a job queue

DRD §6.3 calls Temporal “strongly recommended.” This document upgrades that to a firm recommendation, because three requirements in DRD §5.5 are extremely difficult to satisfy with a conventional queue (Celery, BullMQ, Cloud Tasks):

1. **Every automated change must be logged before and after execution, and be reversible.** With a queue, a worker that dies between “log intended change” and “execute mutation” leaves an orphaned record and no way to know whether Google applied the change. Temporal's durable execution makes the log-mutate-log sequence a replayable unit: on worker crash the workflow resumes at the exact step, and activity idempotency keys prevent double application.
2. **Guardrail checks must be provably applied.** Because the guardrail evaluation and the mutation are steps in one workflow with recorded inputs and outputs, the workflow history is the audit evidence that

the guardrail ran with those specific limits. Reconstructing that from queue logs is possible but fragile, and it is exactly the evidence an audit or a merchant dispute requires.

3. **Long-running, multi-day, human-in-the-loop processes.** Approve-First mode means a proposed change may wait days for merchant approval. Temporal models this as a workflow blocked on a signal, with a timer for expiry. In a queue-based system this becomes a bespoke state machine in the database, which is where this class of system usually accumulates its worst bugs.

Cost of this choice. Temporal is a genuinely more complex operational dependency than Redis plus a worker pool, and the deterministic-workflow programming model has a real learning curve (no non-deterministic calls in workflow code; all I/O in activities). Budget one week of team ramp-up. Temporal Cloud removes the burden of self-hosting the cluster and is the recommended deployment. If the team has no Temporal experience and the timeline is fixed, the fallback is Celery + Redis with a hand-written outbox table and idempotency keys — workable, but expect to reinvent a subset of Temporal, less reliably.

3.4 Runtime topology

Independent scaling matters here because the two workloads have opposite shapes. The API tier is latency-sensitive and bursty with user traffic; the Optimization Engine is throughput-sensitive, runs in scheduled waves, and is bound by Google Ads API quota rather than CPU.

Service	Scaling driver	Notes
web (Next.js)	Request concurrency	Cloud Run, min 1 instance
api (FastAPI)	Request concurrency	Cloud Run, min 2 instances
worker-generation	Temporal task-queue depth	Campaign generation; CPU + LLM latency bound
worker-optimization	Temporal task-queue depth	Optimization jobs; quota bound, not CPU bound — see §5.4
worker-crawler	Temporal task-queue depth	Isolated: untrusted third-party content, headless browser, separate egress policy
ads-gateway	Called internally by workers	The only service with Google Ads credentials; centralises rate limiting

The crawler is a separate service specifically because it executes against untrusted merchant websites. Isolating it limits the blast radius of a malicious page or an SSRF attempt, and lets it carry a restrictive egress policy that the rest of the platform does not need. See §12.6.

3.5 Cloud provider

GCP is recommended, but weakly. The genuine advantages are operational adjacency rather than technical: Google Ads API and Google Tag Manager API credentials are managed in the same Google Cloud Console project that PPCWay needs anyway for OAuth, brand verification lives there, and BigQuery is the natural Phase 3 analytics store for Google Ads data (Google publishes a first-party Ads-to-BigQuery transfer service).

Nothing in this architecture is GCP-specific. AWS equivalents:

GCP	AWS
Cloud Run	ECS Fargate or App Runner
Cloud SQL (PostgreSQL)	RDS PostgreSQL / Aurora
Memorystore	ElastiCache
Cloud Storage	S3
Secret Manager + KMS	Secrets Manager + KMS
BigQuery	Redshift or Athena
Cloud Operations	CloudWatch

Recommendation: if A1 Inc.’s existing infrastructure (the “A2 stack”) is on AWS, stay on AWS. Operational consistency and an existing on-call runbook are worth more than GCP adjacency. The one item to keep on Google regardless is the Google Cloud project that owns the OAuth client and the developer token — that is not optional.

4 System Architecture

4.1 Architecture principles

Five principles constrain the design. They exist to make the platform auditable and to contain the two things that can destroy merchant trust: unauthorised spend, and policy violations.

1. **One door to Google.** Exactly one service (ads-gateway) holds Google Ads credentials and is the only code path that may issue a mutate call. Rate limiting, quota accounting, retry/backoff, and guardrail enforcement are implemented once, there.
2. **Deterministic decisions, generative content.** The LLM writes text. It never decides budget, bidding strategy, campaign type, or whether to pause a keyword. Those are rules-engine decisions, expressed as code and covered by unit tests. See §7.1.
3. **Log intent before acting.** Every mutation writes an `OptimizationAction` row with intended before/after values *before* the API call, then updates it with the result. A crash therefore leaves a record marked pending, which reconciliation can resolve — never a silent change.
4. **Every automated change is reversible.** Storing the before-state is what makes an undo possible. Without it, “we paused 3 keywords” is not a reversible action.
5. **Fail closed on spend.** If the guardrail layer cannot evaluate a limit — missing configuration, stale data, unreachable database — the mutation does not execute. The default answer to “may I change this merchant’s spend?” is no.

4.2 High-level architecture



4.3 Component responsibilities

Component	Owns	Explicitly does not own
web	Rendering, client state, marketing SEO pages	Any business logic; any credential

Component	Owns	Explicitly does not own
api	Request auth, RBAC, validation, CRUD, starting workflows, serving cached reports	Calling Google Ads directly; long-running work
worker-generation	Crawl orchestration, keyword themes, LLM copy, structure assembly, preview rendering	Executing mutations (proposes only)
worker-optimization	Scheduled + event-driven optimization jobs, rule evaluation, anomaly detection	Bypassing guardrails; direct Google calls
worker-crawler	Fetching and parsing merchant sites under <code>robots.txt</code> and rate limits	Anything requiring platform credentials
ads-gateway	All Google Ads API I/O, guardrails, quota, retries, idempotency, token refresh	Business rules about <i>what</i> to change
Temporal	Durable orchestration, retries, timers, human-approval signals, workflow history	Storing product data
PostgreSQL	Merchants, links, profiles, campaign metadata, action log, billing, config	High-volume time-series metrics
Metrics store	Daily/hourly performance aggregates for dashboards and rule inputs	System of record for anything

4.4 Flow 1: Onboarding to first live campaign

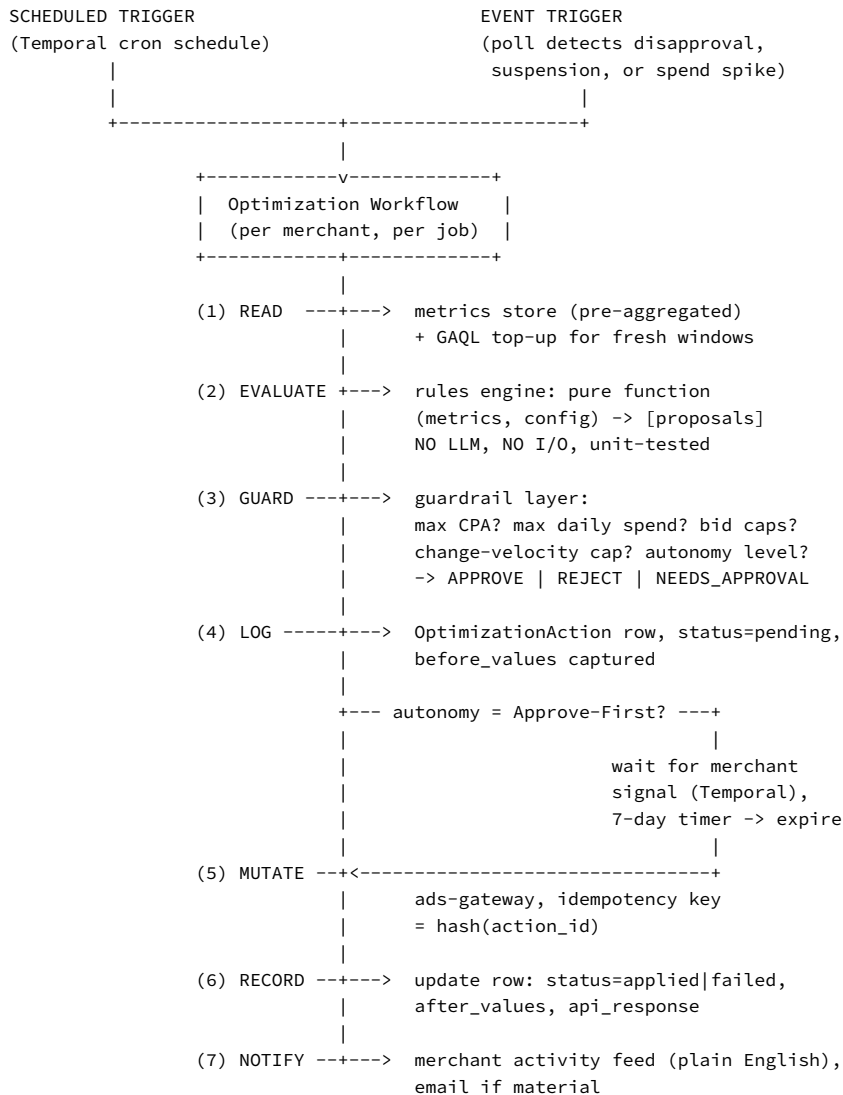
The target from DRD §2 is under 15 minutes median from signup to live campaign. The path below is designed around that, with the one unavoidable external dependency called out explicitly.

STEP	ACTOR	TYPICAL	NOTES
1 Sign up + select plan	merchant / api	45 s	Stripe Checkout
2 Google OAuth consent	merchant / Google	30 s	single consent screen, Ads + GTM scopes together
3 Link account under MCC	ads-gateway	5 s	invitation + accept, or <code>CreateCustomerClient</code> (Ph.2)
4 Business Profile wizard	merchant / api	3-5 min	the main human step
5 Site crawl	worker-crawler	20-60 s	runs DURING step 4
6 Keyword themes	worker-generation	15-40 s	<code>KeywordPlanIdeaService</code> + LLM
7 Ad copy generation	worker-generation	20-60 s	LLM, then policy validator
8 Structure assembly	worker-generation	2 s	deterministic rules engine
9 Plain-English preview	api / web	-	merchant reviews
10 BILLING GATE	merchant / Google	UNBOUNDED	<-- see note below
11 Conversion tracking setup	merchant / api	1-4 min	GTM API auto-config, or manual
12 Approve + launch	ads-gateway	10-20 s	batched mutates
13 Optimization workflow starts	worker-optimization	-	continuous from here

Step 10 is the single largest threat to the 15-minute target, and it is outside PPCWay's control. Google does not permit programmatic entry of payment-card data, and a payments profile can only be created through the Google Ads UI. Any merchant without an existing, valid payment method must leave PPCWay, complete Google's billing flow, and return. Design implications in §13.1.

Because step 10 cannot be removed, the funnel must be built so that everything else is finished before the merchant hits it. Specifically: run the crawl concurrently with the wizard (step 5 overlaps step 4); check billing status via `CustomerService` immediately after linking so the merchant is warned at minute 1 rather than minute 12; and make the campaign preview reviewable and approvable in a state where the only outstanding item is billing, so a merchant who leaves to fix billing can return to a one-click launch.

4.5 Flow 2: The optimization loop



The ordering of steps 3, 4 and 5 is deliberate and load-bearing. Guardrails run before logging so that a rejected proposal is recorded as rejected rather than as an attempted change. Logging runs before mutating so that no mutation can exist without a record. This is what makes the Optimization Action Log trustworthy as an audit artifact, which DRD §5.5 requires.

4.6 Multi-tenancy and isolation

Every tenant-scoped table carries `merchant_id`, and PostgreSQL row-level security is enabled with the application setting `app.current_merchant_id` per request or per workflow. This is defence in depth: a missing `WHERE merchant_id = ...` in application code becomes a query that returns nothing rather than a cross-tenant data leak.

Because the Agency persona (DRD §3.2) needs to act across several merchants, agency access is modelled as an explicit `AgencyMembership` grant that widens the RLS predicate to a set of merchant IDs, rather than as a bypass of RLS. Internal admin access uses a separate role with its own audit trail (DRD §5.9).

4.7 Frontend design constraints

DRD §6.2 asks for a design system that avoids dense “ads platform” aesthetics. Three architectural consequences, so this survives contact with implementation:

1. **The default dashboard renders no Google Ads jargon.** The translation from API entities to plain English is a backend concern, not a frontend one — `api` returns pre-narrated strings, so the same wording is reused by the weekly email digest without duplicating logic. The “Advanced view” toggle exposes raw entities from a separate endpoint.
2. **Mobile is a first-class target, not a breakpoint.** DRD §6.2 notes SMB owners check performance from a phone. The default dashboard must be designed at 390 pt width first.
3. **The campaign preview is a bespoke component, not a form.** It is the merchant’s only opportunity to catch a bad generated campaign before spend starts, and it is where trust is won or lost. It gets its own design and usability pass.

5 Google Ads API Integration Layer

This is the highest-risk subsystem in the platform and the one most likely to be under-estimated. Everything in this section is verified against Google’s published documentation as of August 2026, with the caveat that Google’s limits and access rules change often — §5.8 defines the process for keeping this current.

5.1 Access tiers: what each one permits

The developer token’s access level determines both the daily operation ceiling and which API services are usable at all. This single table is the most important operational fact in the document, because two of the four tiers cannot support PPCWay’s product at all.

Tier	Ops/day (production)	Ops/day (test)	Blocks	Review time
Test Account	not permitted	15,000	All production accounts	immediate
Explorer	2,880	15,000	Account creation, user management, planning services, billing services	auto-granted
Basic	15,000	15,000	nothing	~5 business days
Standard	unlimited	15,000	nothing	~10 business days

Three consequences follow, and each is a product decision rather than an engineering one:

Explorer Access cannot run PPCWay, even in beta. The Explorer tier was introduced in February 2026 to let developers reach production accounts without waiting for human review. It blocks `CustomerService.CreateCustomerClient` (so no Phase 2 account provisioning), the planning services including `KeywordPlanIdeaService` (so no keyword-volume-informed generation — the core of DRD §5.4), and the billing services (so PPCWay cannot even read a merchant’s billing state to warn them). It is useful only for early integration spikes.

Basic Access supports a closed beta, not a launch. The 15,000 ops/day ceiling applies to the *developer token*, i.e. to PPCWay in total across all merchants — not per merchant, not per account. §5.4 works the arithmetic; the short answer is a few hundred merchants.

Standard Access is a launch gate on the critical path. It must be applied for early, because it carries an RMF compliance obligation (§12.3) that shapes UI scope, and because review takes around 10 business days and can bounce back for clarification. Since July 2026 Google runs a pilot in which completing **Google Cloud brand verification** on the linked Cloud project reduces review from days to a few hours for pending applications. A1 Inc. should complete brand verification regardless.

Action for Phase 0. Apply for Basic Access on day 1 (it unblocks real integration work), complete brand verification in parallel, and submit the Standard Access application as soon as there is a demonstrable UI to show — Google’s reviewers assess the actual tool. Do not discover in launch week that the token is still Basic.

5.2 Account structure

PPCWay operates a **Manager Account (MCC)**. Merchant accounts are linked as client accounts beneath it. A manager account can link up to **85,000 non-manager accounts**, which is far beyond Phase 1–3 volume, so a single MCC is correct for now (Assumption A2).

Two link paths must both be supported, because they serve different merchants:

```

PATH A -- LINK AN EXISTING ACCOUNT (Phase 1)
  merchant authorises via OAuth
    -> PPCWay sends a link invitation from the MCC to the customer ID
    -> merchant (or PPCWay on their behalf, with consent) accepts
    -> CustomerClientLink status becomes ACTIVE
    -> PPCWay can now operate on the account via login-customer-id header

PATH B -- CREATE A NEW ACCOUNT (Phase 2)
  merchant authorises via OAuth
    -> CustomerService.CreateCustomerClient under the MCC
    -> new account is created ALREADY LINKED (no invitation step)
    -> merchant must still complete BILLING in Google's own UI <-- unavoidable
    -> PPCWay polls billing_setup / customer.status until serviceable

```

Even though a single MCC is sufficient by account count, every `GoogleAdsLink` row stores the `manager_customer_id` it was linked under. This costs nothing now and makes a Phase 3 migration to a sub-MCC hierarchy (for example, one sub-MCC per agency partner or per region) a data migration rather than a re-architecture.

Note on `CreateCustomerClient`. Since March 2025 this method mirrors the Google Ads UI’s account-creation rules and returns dedicated errors when the *manager* account is below Google’s spending thresholds, and when there is a policy violation. In practice this means a brand-new PPCWay MCC may not be permitted to create client accounts at all until it has accumulated spend history. Phase 2 planning must assume this and verify with Google before committing to a date.

5.3 OAuth and token lifecycle

PPCWay uses OAuth 2.0 with offline access to obtain a refresh token per merchant.

Item	Design
Scopes requested	https://www.googleapis.com/auth/adwords , plus GTM/GA4 scopes only when the merchant opts into automated conversion setup
Consent strategy	One consent screen covering Ads and (optionally) Tag Manager, to avoid a second prompt mid-onboarding. DRD §7 calls for this
Refresh token storage	Encrypted with a per-merchant data key; data key wrapped by Cloud KMS (envelope encryption). See §12.1
Access token cache	Redis, keyed by merchant, TTL slightly under Google’s expiry; never persisted to disk
Revocation detection	A refresh failure with <code>invalid_grant</code> sets <code>GoogleAdsLink.status = REVOKED</code> , pauses all optimization workflows for that merchant, and raises an in-app plus email notification
Least privilege	PPCWay requests only <code>adwords</code> ; it does not request broader Google account scopes. DRD §8.1
Re-consent path	A one-click “reconnect Google Ads” flow that preserves the merchant’s profile and campaign metadata

The revocation path matters more than it first appears. A merchant who revokes access in their Google account settings leaves PPCWay with live campaigns it can no longer manage or pause. The product must

therefore distinguish clearly, in the UI and in the billing logic, between “optimizing” and “connection lost — your campaigns are still spending and we cannot manage them.” Continuing to bill a subscription for optimization that cannot happen is both a support problem and a refund liability.

5.4 Quota model: sizing the platform against the operation ceiling

Quota is consumed **per operation**, not per request, and both reads and writes count. Batching 1,000 keyword additions into one request is one request but one thousand operations. This is the constraint that determines how many merchants PPCWay can serve, so it must be modelled before the optimization cadence is designed — not after.

5.4.1 Estimated steady-state operations per merchant per day

Activity	Cadence	Ops per run	Ops/day
Performance pull (campaign, ad group, keyword, search terms) via GAQL	4x daily	~4	16
Budget pacing check + adjust	daily	1–3	2
Bid / target adjustment	daily (subset of merchants)	2–6	3
Search-term mining (promote + negate)	2x weekly	10–30	6
RSA asset review / rotation	weekly	4–12	1
Policy + suspension + billing poll	2x daily	3	6
Change-event reconciliation	daily	2	2
Steady-state total			~36–40

One-off costs sit on top of this: initial campaign build is roughly 150–400 operations per merchant depending on keyword count and whether Performance Max is included, and it happens once.

5.4.2 What that means per access tier

Access tier	Ops/day budget	Merchants supportable at ~40 ops/day	Verdict
Explorer (2,880)	2,880	~70, minus blocked services	Unusable — blocks planning and billing services
Basic (15,000)	15,000	~300–375	Closed beta only
Standard (unlimited)	governed by system rate limits, not a daily cap	Thousands+	Required for launch

5.4.3 Rate limits that persist even on Standard Access

“Unlimited” removes the *daily* cap. It does not remove system rate limits, and several of these directly shape the design:

Limit	Value	Design consequence
Mutate operations per request	10,000 max	Chunk large builds; exceeding returns <code>TOO_MANY_MUTATE_OPERATIONS</code>
“Action” operations per request	100 max	Separate chunk size for these; do not reuse the 10,000 figure
Keyword-plan idea generation	~1 request/sec per customer ID	Serialise per merchant; parallelise <i>across</i> merchants
Ad-group theme suggestions	~2 requests/sec per customer ID	Same pattern

Limit	Value	Design consequence
Planning services generally	~1 QPS per customer ID	Generation cannot be made faster by adding workers to one merchant
Audience Insights	~200 requests/day per customer ID	Sufficient for onboarding, not for continuous polling
Conversion upload	2,000 conversions per request	Relevant only if offline conversion import is added later
Billing / account-budget mutate	1 operation per request	No batching available here
gRPC response size	64 MB	Use SearchStream for large reports; page and narrow field selection

The important structural insight: the per-customer-ID limits mean the Optimization Engine scales by **fan-ning out across merchants**, never by adding concurrency within one merchant. This is why worker-optimization is described in §3.4 as quota-bound rather than CPU-bound, and why the Temporal task queue is partitioned by merchant.

5.4.4 Quota accounting implementation

ads-gateway maintains three token buckets in Redis and refuses to issue a call that would breach any of them:

1. **Global daily operations** — a counter keyed to the developer token, reset on Google's daily boundary. Alerts fire at 60%, 80% and 92% consumption (DRD §5.9 requires alerting before limits are hit).
2. **Per-customer-ID QPS** — a leaky bucket per merchant for the planning services.
3. **Per-service concurrency** — a semaphore preventing a single job type from starving others.

Non-urgent work (weekly RSA review, search-term mining) carries a lower priority class and is deferred when the global bucket crosses 80%. Merchant-visible and safety-critical work (budget pacing, disapproval detection, an explicit merchant-triggered action) is never deferred. Without this priority split, a quota squeeze degrades exactly the wrong things.

5.5 The guardrail layer

This is the component that stands between an automated decision and a merchant's money. It is implemented once, inside ads-gateway, and it is the only code path permitted to call a mutate method. Any service that could bypass it is a defect.

Every proposed mutation is submitted as a structured object and evaluated against the merchant's configuration:

```
GUARDRAIL EVALUATION ORDER (first failure short-circuits)

1. LINK VALID?           GoogleAdsLink.status == ACTIVE, token refreshable
2. ACCOUNT SERVICEABLE? not SUSPENDED, not CANCELED, billing not on hold
3. AUTONOMY LEVEL?      FULL_AUTO    -> may apply
                        APPROVE_FIRST -> must queue for merchant approval
                        ALERTS_ONLY   -> must not apply; notify only
4. SCOPE?               entity belongs to this merchant's customer_id
                        (defence against a cross-tenant logic error)
5. HARD SPEND LIMITS?   new daily budget <= merchant.max_daily_budget
                        target CPA     <= merchant.max_cpa
                        bid              within [min_bid, max_bid]
6. DELTA LIMITS?        single change  <= max_change_pct (default 25%)
                        cumulative today <= max_daily_change_pct (default 50%)
7. VELOCITY LIMITS?     changes to this entity in last 24h <= N
                        total changes for merchant today <= M
8. BLAST RADIUS?        proposed pause count <= X% of active entities
                        (prevents "pause everything" from a bad metrics read)
9. DATA FRESHNESS?     metrics backing this decision are < staleness_threshold
                        (a stale read must not drive a spend change)
10. QUOTA?              global + per-CID buckets have headroom

RESULT: APPROVED | REJECTED(reason) | REQUIRES_APPROVAL
FAIL-CLOSED: any check that cannot be evaluated returns REJECTED.
```

Four of these deserve emphasis because they are the ones usually left out and are the ones that cause incidents:

- **Check 8, blast radius.** The realistic catastrophic failure is not a slightly wrong bid; it is a metrics pipeline returning zeros and the rules engine concluding that every keyword is non-converting. Capping the proportion of entities that any single job may pause turns a potential account-wide outage into a small, visible anomaly.
- **Check 9, data freshness.** A spend decision made on stale data is worse than no decision. If the metrics store has not been refreshed within the threshold, the job defers rather than acts.
- **Check 6, cumulative delta.** A per-change cap alone is insufficient: twenty compliant 20% increases in a day is a 38x budget increase. The daily cumulative cap is what actually bounds exposure.
- **Check 4, scope.** This is redundant with row-level security and application logic. It stays because the cost of the redundancy is one comparison and the cost of the failure it prevents is spending merchant A's money on merchant B's campaign.

Guardrail limits are versioned. Each OptimizationAction records the guardrail_config_version that authorised it, so that “which limits were in force when this change was made?” is answerable months later, which is what a merchant dispute actually requires.

5.6 Idempotency, retries and backoff

DRD §8.4 requires mutations to be idempotent and retry-safe. Three mechanisms combine:

1. **Idempotency keys.** Every mutation carries a deterministic key derived from the OptimizationAction UUID. ads-gateway records the key with its outcome before returning. A Temporal activity retry with the same key returns the recorded result instead of re-issuing the call.

2. **validate_only pre-flight.** For campaign creation and any mutation touching more than a handful of entities, the gateway first issues the request with `validate_only = true`. This catches schema and policy errors without consuming a real write, and turns a partially-applied build into a caught error.
3. **partial_failure handling.** For batched mutations the gateway uses `partial_failure` so a single bad keyword does not reject the batch, then parses the per-operation error list and records precisely which operations succeeded. Silently discarding the partial-failure payload is a common and expensive integration bug: the platform believes it created 400 keywords when it created 386.

Retry policy by error class:

Error	Retryable	Strategy
RESOURCE_EXHAUSTED (daily quota)	No	Stop the job class; alert; resume next quota day
RATE_EXCEEDED / per-CID throttle	Yes	Exponential backoff with jitter, cap 5 attempts
INTERNAL / UNAVAILABLE / DEADLINE_EXCEEDED	Yes	Exponential backoff with jitter, cap 5 attempts
AUTHENTICATION_ERROR (token)	Once	Force refresh, retry once, then mark link REVOKED
AUTHORIZATION_ERROR	No	Link is broken; surface to merchant
Validation / policy errors	No	Record on the action; surface in plain English
TOO_MANY_MUTATE_OPERATIONS	No	Programming error; re-chunk and alert engineering

Reconciliation job. Any `OptimizationAction` left in pending for longer than its expected duration is reconciled against the Google Ads `change_event` stream to determine whether the mutation actually landed. This closes the window where PPCWay crashed after Google applied a change, which is precisely the state that an “immutable, reversible log” must not get wrong.

5.7 Detecting disapprovals, suspensions and billing holds

DRD §6.4 correctly notes that the Google Ads API does not push these as webhooks. Detection is by polling, and the cadence is a direct trade-off against quota.

Signal	Source	Cadence	Ops
Ad / asset policy disapproval	GAQL on <code>ad_group_ad</code> and asset policy summaries	2x daily	1–2
Keyword disapproval	GAQL on <code>ad_group_criterion</code> approval status	daily	1
Account suspension	<code>customer.status</code>	2x daily	1
Billing hold / payment failure	<code>billing_setup</code> , plus customer serving-eligibility diagnostics	2x daily	1
Unexpected external change	<code>change_event</code> (changes not originated by PPCWay)	daily	1–2
Budget exhaustion / under-delivery	campaign metrics vs. budget; budget recommendations	4x daily	see above

The change-event poll deserves separate mention, because it detects changes that a merchant or their agency made *directly in Google Ads*, outside PPCWay. Without it, the engine and a human can each undo the other’s work, and the merchant sees instability nobody can explain. When an external change is detected, the correct behaviour is to reconcile PPCWay’s cached state and notify rather than to revert.

All of these feed the same event bus, which triggers ad-hoc optimization workflows (DRD §6.6) and merchant notifications in plain English (DRD §5.8).

5.8 API version upgrade process

Since January 2026 the Google Ads API ships a new major version roughly monthly, with four major versions a year and a **12-month support window** per version. The current version as of this document is **v25** (released 22 July 2026). This is a permanent maintenance obligation, and it must be scheduled work rather than an emergency.

Standing process:

1. Pin the API version explicitly in configuration. Never track “latest” implicitly.
2. Subscribe the engineering team to the Google Ads Developer Blog release announcements.
3. On each major release, run a scheduled review task: read the migration notes, run the integration test suite against the new version in the test-account environment, and record a go/no-go.
4. Upgrade at most one major version behind the newest, and never allow the pinned version to come within 90 days of its sunset date.
5. Treat any removed or renamed field as a schema change and version the internal mapping layer.

Budget **2–4 engineer-days per quarter** steady-state, with occasional larger spikes when Google makes a breaking change to a service PPCWay depends on heavily. This line item is easy to omit from a project plan and impossible to omit from reality.

5.9 Services used, mapped to features

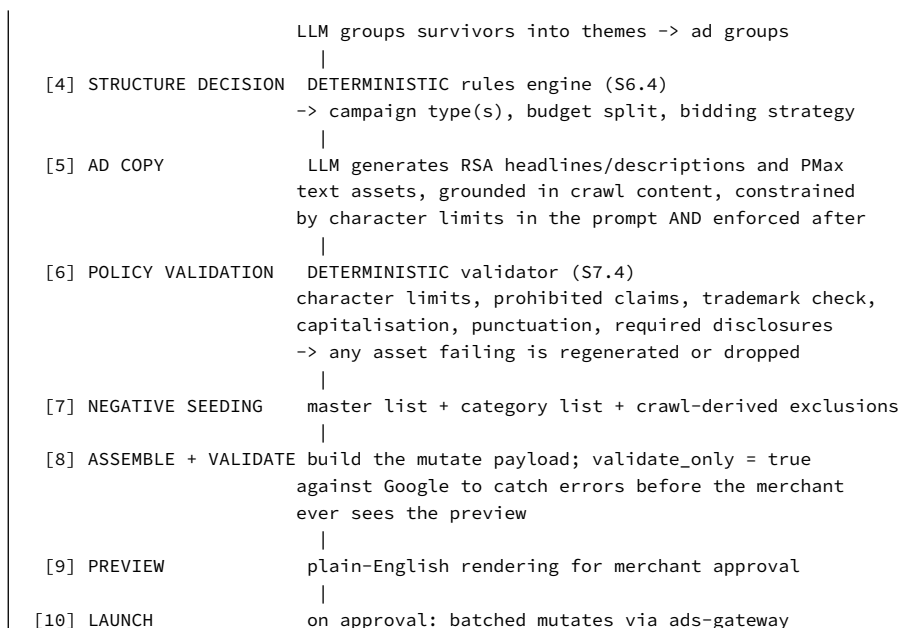
Google Ads API service	Used for	DRD reference
CustomerService	Account creation (Phase 2), account metadata, serviceability checks	§5.2
CustomerClientLinkService	Linking existing merchant accounts to the MCC	§5.2
GoogleAdsService (Search / SearchStream)	All reporting via GAQL; change-event and policy polling	§5.6, §6.4
CampaignService, CampaignBudgetService	Campaign creation, status, budget changes	§5.4, §5.5
CampaignCriterionService	Location, language, negative keywords at campaign level	§5.4
AdGroupService, AdGroupCriterionService	Ad groups, keywords, keyword-level negatives, bids	§5.4, §5.5
AdGroupAdService	RSA creation, pausing, rotation	§5.4, §5.5
AssetService, AssetGroupService, AssetGroupAssetService	Performance Max text, image and logo assets	§5.4
KeywordPlanIdeaService	Keyword theme generation with volume and competition data	§5.4
ConversionActionService	Creating and validating conversion actions	§5.3
SharedSetService, SharedCriterionService	Shared negative keyword lists	§5.4
BiddingStrategyService	Portfolio bidding strategies where applicable	§5.4
RecommendationService	Optional input to the optimization rules engine	§5.5
BillingSetupService (read)	Detecting whether billing is complete	§5.2
GoogleAdsFieldService	Schema introspection used by the version-upgrade test suite	§5.8

6 Campaign Generation Engine

6.1 Pipeline

Generation is a Temporal workflow. Each step is an activity with its own retry policy, so a transient LLM or crawl failure does not restart the whole pipeline.

[1] CRAWL	robots.txt-respecting fetch of up to N pages extract: business name, services/products, USPs, phone, address, meta/title, page-speed + mobile signals
[2] NORMALISE	 deterministic extraction into a BusinessProfile record (LLM used only for summarising free text, never for deciding structure)
[3] KEYWORD THEMES	 KeywordPlanIdeaService seeded from crawl + category filter by volume, competition, and relevance



Step 8 is easy to skip and valuable to keep. Running `validate_only` *before* showing the preview means the merchant never approves a campaign that then fails to launch — which is the worst possible moment to discover a validation error, because it destroys confidence at exactly the point where the merchant has committed.

6.2 Website crawl

Aspect	Design
Compliance	robots.txt honoured; descriptive user agent identifying PPCWay with a contact URL
Politeness	Max 1 concurrent request per host, 1 request/sec, max 25 pages, 60 s total budget
Rendering	Static fetch first (httpx + selectolax); escalate to headless Chromium only if the static parse yields insufficient text
Extraction	Structured data first (JSON-LD, microdata, OpenGraph), then headings and body text, then tel:/mailto: and address patterns
Quality signals	Page weight, render-blocking resources, viewport meta, HTTPS, Core Web Vitals via PageSpeed Insights API — feeds Quality-Score-oriented advice per DRD §5.3
Failure modes	If the crawl yields too little (JS-only SPA, login wall, blocked), the wizard falls back to asking the merchant directly rather than generating from thin data
Storage	Raw HTML retained in object storage for 30 days for debugging and re-generation; extracted profile retained for the account lifetime
Security	Runs in the isolated <code>worker-crawler</code> service. SSRF defences in §12.6

That fallback in row 6 is not an edge case. A meaningful share of SMB sites are JS-rendered, behind Wix/Squarespace patterns, or thin enough that no useful copy can be extracted. The wizard must degrade gracefully to manual entry instead of generating plausible-sounding ad copy from nothing — which is the failure mode that produces embarrassing live ads.

6.3 Keyword theme generation

Deterministic filtering happens *before* the LLM sees anything, so the LLM groups a pre-qualified set rather than inventing keywords:

1. Seed `KeywordPlanIdeaService` with crawl-derived terms plus the mapped business category, and the merchant's geo targeting.

2. Retrieve ideas with `avg_monthly_searches`, `competition`, and the bid range.
3. Filter deterministically: minimum volume threshold scaled to the merchant's budget, exclude competitor brand terms by default (trademark risk, §7.4), exclude the master negative list, exclude terms whose implied CPC would exhaust the daily budget in fewer than ~3 clicks.
4. Ask the LLM to group the survivors into 3–8 tightly-themed clusters and name each one.
5. Validate deterministically that every returned keyword appeared in the input set. **The LLM is not permitted to introduce keywords.** Any hallucinated term is dropped, and a high hallucination rate raises an alert.

Match-type strategy at launch: phrase and exact for the highest-confidence terms, broad only where the budget can absorb the learning cost, paired with aggressive negative seeding.

6.4 Structure decision rules engine

This is deliberately **not** LLM-driven. DRD §6.5 requires it to be auditable and testable independently of LLM output, and a merchant's budget allocation is not a language problem. It is a pure function: `(BusinessProfile, budget, goal, conversion_state) -> CampaignPlan`, covered by a table-driven test suite.

Business type	Goal	Monthly budget	Plan
Local service	Calls / leads	< \$750	1 Search campaign, geo-radius, call asset, call-conversion tracking
Local service	Calls / leads	\$750–3,000	Search + Local; call and form conversions
Local service	Store visits	any	Search + Local, store-visit conversions where eligible
E-commerce	Online sales	< \$1,000	Search on high-intent product/category terms only
E-commerce	Online sales	\$1,000–5,000	Search (brand + high intent) + Performance Max with retail feed if present
E-commerce	Online sales	> \$5,000	Search + PMax, split by margin tier or product category
Professional practice	Form leads	any	Search only; tight phrase/exact; strong negatives (high CPC verticals)
Any	App installs	any	Out of v1 scope — App campaigns need their own asset pipeline

Budget split rule when two campaign types are used: Search receives 60–70% initially, because its performance is attributable and diagnosable, and Performance Max receives the remainder until it has enough conversion volume to justify a larger share. PMax is deliberately not the default first campaign for a small budget: it is a black box that is hard to explain in plain English, and explicability is a core product promise here.

6.5 Bidding strategy decision table

Bidding strategy is chosen from conversion data availability, not from preference. Applying Target CPA before the account has conversion history is one of the most common ways an automated tool wastes a merchant's budget.

Conversions in trailing 30 days	Budget	Strategy	Rationale
0 (new account, tracking verified)	any	Maximize Clicks with a bid cap	Gather traffic data; the cap bounds waste
0 and tracking not verified	any	Do not launch	§13.5 — launching blind is the failure this product must avoid

Conversions in trailing 30 days	Budget	Strategy	Rationale
1–14	any	Maximize Conversions, no target	Not enough data for a reliable target
15–29	any	Maximize Conversions	Approaching threshold; monitor
30+	any	Target CPA, seeded at trailing CPA	Sufficient volume for a target
30+ with revenue values	any	Target ROAS, seeded at trailing ROAS	Value-based bidding once values are trustworthy
50+ and stable for 14 days	any	Target CPA/ROAS with tightened target	Optimize rather than learn

Every transition is recorded in the Optimization Action Log with its trigger, and every transition resets a learning-period timer during which the engine makes no further bidding changes for that campaign. Without that timer the engine fights Google’s own learning phase and both perform worse.

6.6 Negative keyword strategy

Three layers, applied at build time:

1. **Master list** (platform-wide): jobs/careers/salary, free/cheap/DIY, tutorial/how-to, pornographic and illegal terms, competitor-complaint patterns, “reviews”/“complaints” where inappropriate for the goal.
2. **Category list**: maintained per business category. For example, a plumber excludes “plumbing courses”, “plumber salary”, “plumbing supplies”; a law firm excludes “free legal advice”, “law school”.
3. **Crawl-derived**: services the merchant explicitly does *not* offer, detected from the site (an “we do not service X” statement, or a services list that excludes an obvious adjacent term).

Shared negative lists are used where possible so a single update propagates. Note the platform limit: **20 negative keyword lists per manager or child account**, and **10,000 negative keywords per campaign** — generous, but the list count is not, so lists must be composed thoughtfully rather than created per-merchant-per-theme.

6.7 Ad copy generation and creative constraints

Character limits are enforced twice: stated in the prompt, and validated deterministically afterwards. Prompt-only enforcement is unreliable; validator-only enforcement wastes tokens on regeneration. Both is correct.

6.7.1 Responsive Search Ads

Asset	Count	Limit
Headlines	3 min, 15 max (target 10–12)	30 characters
Descriptions	2 min, 4 max (target 4)	90 characters
Display path fields	0–2	15 characters each
Enabled RSAs per ad group	3 max	platform limit

6.7.2 Performance Max asset groups

Asset	Minimum	Maximum	Limit
Headlines	3	15	30 characters
Long headlines	1	5	90 characters
Descriptions	2	5	90 chars (one should be <= 60)

Asset	Minimum	Maximum	Limit
Business name	1	1	25 characters
Logos (1:1)	1	5	min 128x128 px
Landscape images (1.91:1)	1	shared 20-image pool	min 600x314 px
Square images (1:1)	1	shared 20-image pool	min 300x300 px
Videos	0	15	Google auto-generates if absent

Verify these at build time. Google adjusts asset minimums, maximums and recommended counts without notice — the video maximum, for instance, moved from 5 to 15 during early 2026. The validator must therefore read its limits from configuration, and the integration test suite must assert them against the live API rather than hard-coding them in application logic.

Image assets are sourced in priority order: merchant upload, then crawl-extracted images that pass a minimum-dimension and aspect-ratio check, then a licensed stock fallback keyed to the business category. PPCWay does **not** generate images with a model in v1: the licensing, brand-accuracy and misleading-imagery risks are not worth the benefit at MVP, and Google will auto-generate assets when they are absent.

7 LLM Strategy and Selection

7.1 Where the LLM is used, and where it must not be

The single most important design rule in this document. The LLM is a text generator inside a deterministic system, not a decision-maker.

Task	LLM?	Why
Summarise crawled site content into a profile	Yes	Unstructured-to-structured language work
Group pre-filtered keywords into themes	Yes	Semantic clustering; output validated against the input set
Write RSA headlines and descriptions	Yes	The core generative task
Write PMax text assets	Yes	Same
Write the plain-English campaign preview	Yes	Translating entities into merchant language
Write the plain-English weekly digest	Yes	Narrative from structured metrics
Write plain-English optimization-log entries	Yes	Narrative from structured actions
Suggest sitelink and callout text	Yes	Short-form copy
Choose campaign type	No	Rules engine, §6.4
Set or change budget	No	Rules engine + guardrails
Choose bidding strategy	No	Decision table, §6.5
Decide to pause a keyword or ad	No	Rules engine on metrics
Adjust a bid or CPA target	No	Rules engine + guardrails
Judge policy compliance	No	Deterministic validator, §7.4
Introduce new keywords	No	Only KeywordPlanIdeaService output is eligible

Three reasons this boundary is non-negotiable. First, DRD §6.5 explicitly requires the structure logic to be auditable and testable independently of LLM output. Second, spend decisions must be reproducible and explainable to a merchant, and “the model decided” is not an explanation. Third, an LLM asked to judge its own policy compliance is the weakest possible control precisely where the consequence — account suspension — is most severe.

7.2 Model selection

Recommendation: Claude Opus 5 (c`laude-opus-5`) as the default for every generative step.

The reasoning is that ad copy quality is not a cosmetic concern here; it is the product. Ad strength affects auction eligibility and cost per click, generated copy goes live under a merchant's brand with money behind it, and policy-adjacent phrasing mistakes risk disapprovals and, repeated, account suspension. The generative step is also a small fraction of total cost per merchant (\$7.7), so the highest-capability model is the right default and the cost argument for a weaker one is weaker than it first appears.

Current Anthropic models, for reference:

Model	Model ID	Context	Input \$/MTok	Output \$/MTok
Claude Fable 5	<code>claude-fable-5</code>	1M	10.00	50.00
Claude Opus 5	<code>claude-opus-5</code>	1M	5.00	25.00
Claude Sonnet 5	<code>claude-sonnet-5</code>	1M	3.00	15.00
Claude Haiku 4.5	<code>claude-haiku-4-5</code>	200K	1.00	5.00

Notes on the alternatives, so the choice is informed rather than assumed:

- **Claude Fable 5** is the most capable model available, and is aimed at long-horizon agentic and hard-reasoning work. Ad-copy generation is neither, so the premium over Opus 5 does not buy a proportional gain here. It is worth evaluating for the structure-planning step *if* A1 Inc. ever moves that step away from a deterministic rules engine — which this document recommends against.
- **Claude Sonnet 5 and Claude Haiku 4.5** are the cost-down options. \$7.7 quantifies the saving. The trade-off is a genuine product decision for A1 Inc. rather than an engineering one, so it is presented there with numbers rather than decided here.

Required API usage patterns for the chosen models, which differ from older Claude models and will cause hard failures if carried over from prior code:

Pattern	Requirement
Sampling parameters	temperature, top_p, top_k are rejected with HTTP 400 on Opus 5 and Sonnet 5. Steer with prompting, not sampling
Thinking	Use thinking: {type: "adaptive"}. The older budget_tokens form returns 400
Effort	Control depth via output_config: {effort: ...} — low for the narrative tasks, medium/high for ad copy
Assistant prefill	Not supported; returns 400. Use structured outputs instead
Structured output	output_config: {format: {type: "json_schema", schema: ...}} — use this for every generation step so parsing cannot fail
Refusals	Check stop_reason == "refusal" before reading content. A refusal must degrade to a template, never to a crash
Long outputs	Stream any request with a large max_tokens to avoid HTTP timeouts

7.3 Prompt architecture and caching

Prompt caching is where the LLM cost model is won. Caching is a **prefix match**: any byte change invalidates everything after it, so the prompt must be assembled stable-part-first.

```
PROMPT LAYOUT (render order: tools -> system -> messages)

[ STABLE PREFIX -- cached, identical for every merchant in a category ]
  system:  role definition
           Google Ads RSA/PMax policy constraints and character limits
```

```

        prohibited-claim rules, capitalisation and punctuation rules
        writing guidelines and worked examples (good and bad)
        category-specific guidance
        <-- cache_control breakpoint here
[ VOLATILE SUFFIX -- never cached ]
    user:    this merchant's BusinessProfile, crawl extract, keyword themes,
            geo, goal, budget band

```

- Cache read is roughly 0.1x input price; cache write is 1.25x (5-minute TTL) or 2x (1-hour). With a shared prefix of a few thousand tokens across many merchants, this is the difference between a material LLM bill and a negligible one.
- The minimum cacheable prefix is **512 tokens on Opus 5** and **1,024 on Sonnet 5**. Below that, nothing caches and no error is raised — verify via `usage.cache_read_input_tokens` rather than assuming.
- **Silent invalidators to prohibit in code review:** any timestamp, UUID, request ID or merchant name in the system prompt; non-deterministic JSON serialisation (always sort keys); conditionally-assembled system sections. Each of these silently drops the cache hit rate to zero while looking correct.
- Non-urgent generation (weekly digests for thousands of merchants) should use the **Batch API** at 50% of standard price. Digests are a scheduled, latency-insensitive batch — an ideal fit.

7.4 The deterministic policy validator

The gate between generated text and Google. Every check is code, is unit-tested, and produces a machine-readable reason on failure. No model judgement is involved.

Check	Rule
Character limits	Per the tables in §6.7, counted in the same way Google counts (careful with non-ASCII)
Required asset counts	Minimums met for the ad type before submission
Prohibited superlatives	Unsubstantiated “best”, “#1”, “guaranteed”, “cheapest” without evidence
Prohibited claims	Health, financial-return, legal-outcome and similar restricted claims, by category
Trademark terms	Screened against a maintained list; competitor brand terms blocked by default
Capitalisation	No ALL CAPS words (except recognised acronyms); title-case consistency
Punctuation	No repeated exclamation marks; no exclamation in headlines; no unsupported symbols
Phone numbers in text	Blocked — must use a call asset instead
Price and offer claims	Blocked unless present on the crawled landing page (substantiation)
Landing-page consistency	Every headline claim must have supporting text on the destination URL
Restricted vertical gate	Category-level block per DRD §5.3, before generation is even attempted
Duplicate assets	No two headlines that are near-identical (hurts ad strength)
Placeholder leakage	Rejects [brand], {keyword}, TODO, Lorem, and unresolved template syntax
Encoding	Rejects control characters, zero-width characters, mismatched quotes

Failure handling: up to two regeneration attempts with the specific violation fed back, then drop the individual asset if the ad still meets its minimum count, then fall back to a category-specific human-written template. The pipeline must never block a merchant’s launch because a model would not produce a compliant headline.

The **landing-page consistency** check deserves emphasis: it is the single most effective defence against the characteristic LLM failure in this domain, which is inventing a plausible service, offer or credential the business does not actually have. Requiring textual support on the crawled page converts that from a live-ad incident into a dropped asset.

7.5 LLM failure modes and mitigations

Failure mode	Mitigation
Hallucinated services, offers or credentials	Landing-page substantiation check; grounding limited to crawl extract
Hallucinated keywords	Output intersected with <code>KeywordPlanIdeaService</code> results; extras dropped
Character-limit overruns	Deterministic validator; regenerate with the violation stated
Policy-violating phrasing	Validator plus Google's own <code>validate_only</code> pre-flight
Refusal (<code>stop_reason: refusal</code>)	Detect before reading content; fall back to template; log for prompt review
Non-JSON output	Structured outputs (<code>json_schema</code>) rather than parsing free text
Latency spike blowing the 15-minute target	Generation is a Temporal activity with a timeout; partial results usable; merchant sees progress
API outage	Circuit breaker; queue and retry; template fallback so onboarding completes
Cost spike from a prompt regression	Per-merchant and per-day token budget caps with alerting; cache-hit-rate monitoring
Silent quality regression after a model or prompt change	Golden-set evaluation suite (§18.3) run before any prompt or model change ships
Prompt injection via crawled site content	Crawl text is delimited and labelled untrusted in the prompt; output is validated regardless; no tool access from the generation call

The last row is a real attack surface, not a theoretical one. Crawled content is attacker-controlled text being placed into a prompt. The mitigation that actually matters is not prompt wording but the fact that the generation call has no tools, no credentials, and no authority — its output is data that must survive an independent validator before it can affect anything.

7.6 Provider abstraction

A single internal interface (`generate_copy`, `group_keywords`, `narrate`) wraps the provider. Prompts live in version-controlled files with an explicit version identifier recorded against every generated asset, so any live ad can be traced back to the exact prompt and model that produced it. That traceability is what makes a quality regression diagnosable.

This is a thin abstraction on purpose. It exists to make provider substitution a bounded task and to enable golden-set testing — not to build a provider-agnostic framework, which would mean giving up provider-specific features like prompt caching, structured outputs and the Batch API that materially affect cost and reliability.

7.7 LLM cost model

Assumptions: shared prefix ~3,000 tokens cached; per-merchant volatile input ~2,000 tokens; outputs as noted; prices per million tokens from §7.2.

7.7.1 One-time cost per merchant onboarding

Step	Model	Input (cached/fresh)	Output	Cost
Profile summary	Opus 5	3,000 / 3,000	800	~\$0.037
Keyword grouping	Opus 5	3,000 / 2,500	1,200	~\$0.044
RSA copy (3 ad groups)	Opus 5	3,000 / 2,000 x3	1,500 x3	~\$0.148
PMax assets (if applicable)	Opus 5	3,000 / 2,000	1,800	~\$0.057
Preview narrative	Opus 5	3,000 / 1,500	900	~\$0.031
Regeneration allowance (~25%)	Opus 5	—	—	~\$0.079
Total per onboarding				~\$0.40

7.7.2 Recurring cost per merchant per month

Step	Model	Cadence	Cost/month
Weekly digest narrative (Batch API, 50% off)	Opus 5	4x	~\$0.06
Optimization-log narration	Opus 5	~30 entries	~\$0.09
Ad-copy refresh / RSA asset replacement	Opus 5	~1x	~\$0.05
Total per merchant per month			~\$0.20

7.7.3 At scale

Active merchants	Onboardings/month	LLM cost/month
100	30	~\$32
500	120	~\$148
2,000	400	~\$560
10,000	1,500	~\$2,600

This is the key number in the section: LLM spend is roughly \$0.20–0.60 per merchant per month. Against a SaaS subscription in the tens of dollars, it is a rounding error. That is why the recommendation is the strongest model rather than the cheapest.

If A1 Inc. wants to reduce it anyway, the options in descending order of saving and ascending order of risk are: (1) run the two narration tasks — digest and optimization log — on Claude Haiku 4.5, cutting recurring cost by roughly 70% with little quality exposure, since these are mechanical transformations of structured data that never reach Google; (2) move keyword grouping to Sonnet 5, saving ~40% on that step; (3) move ad copy to Sonnet 5, saving ~40% of the onboarding cost. Option 3 is the one to weigh carefully, because ad copy is the output that carries the merchant's brand and their money. A/B the golden-set evaluation before making that change, and revisit if per-merchant economics ever get tight.

All figures above are engineering estimates from token-count modelling, not measured production data. Re-baseline with `client.messages.count_tokens()` against the real prompts during Phase 1 before using them for pricing decisions.

8 Optimization Engine

8.1 Job catalogue

Each row is a Temporal workflow with a cron schedule, its own guardrail profile, and its own quota priority class. Cadences are chosen against the quota model in §5.4, not against how often the data changes — checking hourly what changes daily wastes the quota that the rest of the platform needs.

Job	Cadence	Reads	Decides	Priority
Budget pacing	daily 06:00 local	spend vs. budget, month-to-date pace	Adjust daily budget within cap; flag under/over-delivery	High
Bid / target tuning	daily	CPA, ROAS, conversion volume, trailing 14/30d	Adjust target CPA/ROAS, bid caps	High
Bidding-strategy transition	weekly	trailing 30d conversions	Move up the §6.5 decision table	Medium
Search-term mining	Mon + Thu	search term report, conv. by term	Promote converting terms to exact; negate wasteful terms	Medium
RSA asset review	weekly	asset performance labels	Pause LOW assets, request replacements	Low
PMax asset-group review	weekly	asset-group strength, listing groups	Flag weak groups; request assets	Low

Job	Cadence	Reads	Decides	Priority
Negative-list maintenance	weekly	cross-merchant term patterns	Propose additions to category lists (human-reviewed)	Low
Policy / suspension poll	2x daily	approval statuses, customer.status	Raise alerts; trigger remediation workflow	Critical
Billing-state poll	2x daily	billing_setup, serving eligibility	Raise alerts; block launches	Critical
Anomaly detection	4x daily	CTR, CPC, conv. rate, spend velocity	Raise anomaly; trigger corrective workflow	Critical
Change-event reconciliation	daily	change_event	Detect external edits; reconcile cache; notify	Medium
Metrics ingestion	4x daily	GAQL aggregates	(none — pure ingest)	High
Learning-period watchdog	daily	strategy change timestamps	Suppress further changes during learning	High

Critical priority jobs are never deferred for quota, because they are the ones that protect the merchant. Low priority jobs are the first to shed load.

8.2 Rules, not models

Every optimization decision is a pure function of (metrics, merchant_config, entity_state) returning a list of typed proposals. Pure functions are what make this testable, and the test suite is what makes automated spend changes defensible.

Representative rules, with the thresholds that must be tunable per merchant rather than hard-coded:

```
BUDGET PACING
  if month_to_date_spend > pace_target * 1.15 and days_remaining > 5:
    propose decrease_daily_budget(to: pace_corrected_amount)
  if month_to_date_spend < pace_target * 0.80 and impression_share_lost_budget > 0.10:
    propose increase_daily_budget(to: min(pace_corrected, max_daily_budget))
  # impression-share-lost-to-budget is the qualifier that matters: raising the budget
  # when nothing is being lost to budget just buys lower-quality traffic.

SEARCH TERM MINING -- PROMOTE
  if term.conversions >= 2 and term.cpa <= merchant.target_cpa * 1.2
    and term not already an exact keyword:
    propose add_keyword(term, match_type: EXACT, ad_group: best_semantic_match)

SEARCH TERM MINING -- NEGATE
  if term.clicks >= 12 and term.conversions == 0
    and term.cost >= merchant.target_cpa * 1.0:
    propose add_negative_keyword(term, level: CAMPAIGN)
  # click threshold scales with the merchant's conversion rate so low-traffic
  # accounts are not negated into silence on noise.

BID / TARGET TUNING
  if trailing_14d.conversions >= 15:
    if trailing_14d.cpa > target_cpa * 1.25:
      propose decrease_target_cpa(by: min(15%, max_change_pct))
    if trailing_14d.cpa < target_cpa * 0.75 and impression_share < 0.65:
      propose increase_target_cpa(by: min(10%, max_change_pct))

RSA ASSET ROTATION
  if asset.performance_label == LOW and asset.impressions >= 5000
    and ad_group.enabled_assets_of_type > type_minimum:
    propose pause_asset(asset) + request_replacement(asset.type)

ANOMALY -- SPEND SPIKE
  if hourly_spend > 3 * trailing_28d_same_hour_median and hourly_spend > $10:
    raise anomaly(SPEND_SPIKE); if autonomy == FULL_AUTO
      propose cap_daily_budget(to: merchant.max_daily_budget)

ANOMALY -- CONVERSION COLLAPSE
  if trailing_3d.conversions == 0 and trailing_28d.daily_conversions_median >= 2:
    raise anomaly(CONVERSION_TRACKING_SUSPECTED)
    # deliberately does NOT propose bid changes -- the most likely cause is a
    # broken tag, and optimizing against broken data is how accounts get wrecked.
```

The conversion-collapse rule illustrates the design philosophy for the whole engine: when the data becomes untrustworthy, the correct automated action is to stop and escalate, not to optimize harder.

8.3 Autonomy levels

DRD §5.5 requires three configurable levels. The distinction must be enforced in the guardrail layer (§5.5), not in the UI, so that no code path can act above the merchant's chosen level.

Level	Behaviour	Phase	Consent requirement
Alerts-Only	Detect and notify. No mutations at all except merchant-initiated ones	1	Standard terms

Level	Behaviour	Phase	Consent requirement
Approve-First	Propose; hold as <code>REQUIRES_APPROVAL</code> ; apply only on explicit merchant approval; expire after 7 days	1 (default and only auto mode in Phase 1)	Explicit disclosure of what may be proposed
Full-Auto	Apply within guardrails without per-change approval	2	Separate, explicit, recorded consent naming the change classes, limits, and the merchant's right to revert

The Full-Auto consent record is a compliance artifact, not a checkbox. Google Ads API policy requires user control and transparency for automated changes, and DRD §8.2 requires clear merchant-facing disclosure matching the configured autonomy level. Store: timestamp, IP, the exact consent text version shown, the guardrail limits in force at consent time, and the merchant user who granted it.

A useful intermediate that costs little to build: **Full-Auto with exclusions**, where a merchant enables automation generally but pins specific campaigns, budgets or keywords as manual-only. This is frequently the difference between a cautious merchant adopting automation and refusing it.

8.4 The Optimization Action Log

The audit spine of the platform, and the artifact that makes DRD §5.5's “immutable, versioned and reversible” requirement real.

Property	Implementation
Append-only	No UPDATE except the single pending → applied/failed/rejected transition; enforced by trigger and by a restricted DB role
Written before acting	Row exists with <code>before_values</code> captured prior to the API call
Reversible	<code>before_values</code> and <code>after_values</code> stored as JSON, enough to reconstruct an inverse mutation
Attributable	Records the rule ID, rule version, <code>guardrail_config_version</code> , autonomy level, triggering job run, and the metrics snapshot that justified it
Explainable	Carries both a machine-readable payload and a plain-English narrative for the merchant activity feed
Queryable	Indexed by merchant, campaign, action type and timestamp, for both merchant UI and internal support
Retained	7 years (dispute and audit horizon), archived to cold storage after 18 months

Reversibility in practice: an undo endpoint reads `before_values`, constructs the inverse mutation, submits it through the same guardrail layer as a new action linked to the original, and records it. Undo is a first-class action with its own log entry, not a silent restoration — because an undo that leaves no trace breaks the audit property the log exists to provide.

Reversibility is bounded, and the product must say so. Structural changes can be reverted; the spend that already occurred cannot. Google's own learning-phase state cannot be rewound. The UI must not imply that undo restores the account to its prior *performance*, only to its prior *configuration*.

8.5 Anomaly detection approach

Phase 1–2 uses simple, explainable statistics deliberately: trailing-median comparison with day-of-week normalisation, and thresholds expressed as multiples. This is chosen over a learned model because an alert a support agent cannot explain to a merchant is worse than no alert, and because seasonality in SMB accounts on small budgets produces high variance that a naive model over-fits.

Anomaly	Signal	Automated response
Spend spike	Hourly spend vs. 28-day same-hour median	Cap budget (Full-Auto); alert always
Spend collapse	Zero spend where non-zero expected	Check suspension, billing, policy state; alert
CTR drop	7-day CTR vs. 28-day, per campaign	Check disapprovals; flag creative review
Conversion collapse	3-day conversions vs. 28-day median	Suspect tracking, not performance. Alert; suppress bid changes
CPA spike	7-day CPA vs. target	Tighten target within guardrails; alert
Disapproval surge	Count of newly disapproved entities	Remediation workflow; alert
Quality-score drop	Average quality score by campaign	Flag landing page / relevance review

Phase 3 may add seasonality decomposition and a learned baseline per DRD §11, but only where the output remains explainable in one sentence to a merchant.

9 Data Model

9.1 Core relational schema

PostgreSQL. Every tenant-scoped table carries `merchant_id` with row-level security (§4.6). The DRD §9 entity list is implemented as follows, expanded where the requirements imply more structure than the summary shows.

```

merchant
  id (uuid pk), business_name, legal_name, category_code, website_url,
  country, timezone, currency, autonomy_level, status,
  guardrail_config_id (fk), created_at, updated_at

user_account
  id, email, name, auth_provider_subject, mfa_enabled, status, last_login_at

merchant_membership          -- users <-> merchants, with roles
  merchant_id (fk), user_id (fk), role (OWNER|TEAM_MEMBER|AGENCY_VIEWER),
  invited_by, accepted_at

agency
  id, name, owner_user_id, white_label_config (jsonb, Phase 3)

agency_membership            -- agency <-> merchants (DRD S3.2)
  agency_id (fk), merchant_id (fk), permission_level, granted_at, revoked_at

google_ads_link
  id, merchant_id (fk), google_customer_id, manager_customer_id,
  link_type (LINKED_EXISTING|CREATED_BY_PPCWAY), link_status,
  oauth_token_ref (KMS key id + ciphertext ref, never plaintext),
  scopes_granted (text[]), billing_status, account_status,
  last_verified_at, revoked_at

business_profile
  id, merchant_id (fk), version, primary_goal, monthly_budget_cents,
  service_area (jsonb: radius | regions | shipping zones),
  target_audience_notes, crawl_summary (jsonb), crawl_completed_at,
  crawl_source (CRAWLED|MANUAL|HYBRID), page_speed_signals (jsonb)

conversion_action
  id, merchant_id (fk), google_conversion_action_id, name, category,
  source (SITE_TAG|GTM|GA4_IMPORT|CALL|IMPORT), status,
  verified_at, last_conversion_at, primary_flag

campaign
  id, merchant_id (fk), google_campaign_id, name, campaign_type,
  status, daily_budget_cents, google_budget_id,
  bidding_strategy_type, target_cpa_cents, target_roas,
  learning_period_until, created_by (GENERATED|MERCHANT|EXTERNAL),
  generation_run_id (fk)

ad_group / ad / keyword / asset_group / asset
  mirror the Google hierarchy with google*_id, status, and
  last_synced_at; these are a CACHE of Google state, never the
  system of record

generation_run
  id, merchant_id (fk), workflow_id, status,
  prompt_version, model_id, keyword_source_run_id,
  validation_report (jsonb), preview_payload (jsonb),
  approved_by_user_id, approved_at, launched_at

optimization_action          -- the audit spine (S8.4)
  id (uuid pk), merchant_id (fk), campaign_id (fk, nullable),
  entity_type, google_entity_id,
```

```

    action_type, rule_id, rule_version,
    guardrail_config_version, autonomy_level_at_decision,
    metrics_snapshot (jsonb),
    before_values (jsonb), after_values (jsonb),
    status (PENDING|APPLIED|FAILED|REJECTED|REQUIRES_APPROVAL|EXPIRED|REVERTED),
    rejection_reason, api_error (jsonb), idempotency_key (unique),
    narrative_text, reverted_by_action_id (fk, nullable),
    proposed_at, decided_at, applied_at

guardrail_config                                -- versioned; never mutated in place
id, merchant_id (fk, nullable = platform default), version,
max_daily_budget_cents, max_cpa_cents, min_bid_cents, max_bid_cents,
max_change_pct, max_daily_change_pct, max_changes_per_entity_per_day,
max_pause_pct_per_run, metrics_staleness_seconds,
effective_from, effective_to

autonomy_consent                                -- compliance artifact (S8.3)
id, merchant_id (fk), user_id, autonomy_level, consent_text_version,
guardrail_config_version, granted_at, ip_address, revoked_at

subscription
id, merchant_id (fk), plan_tier, status,
stripe_customer_id, stripe_subscription_id,
current_period_end, cancel_at, trial_ends_at

notification_event
id, merchant_id (fk), type, channel (IN_APP|EMAIL), severity,
payload (jsonb), narrative_text, status, sent_at, read_at

notification_preference
merchant_id (fk), user_id, event_type, channel, enabled

restricted_category_check
id, merchant_id (fk), category_code, decision (ALLOW|BLOCK|REQUIRES_CERT),
required_certification, checked_at, override_by_admin_id

admin_audit_log                                -- DRD S5.9
id, admin_user_id, merchant_id (fk, nullable), action, target,
before_values (jsonb), after_values (jsonb), reason, created_at

api_quota_ledger                                -- DRD S5.9 quota monitoring
date, developer_token_hash, operations_used, operations_by_service (jsonb),
peak_hour, throttle_events

```

Two modelling decisions worth stating explicitly:

- **The campaign/ad group/keyword tables are a cache, not the system of record.** Google Ads is authoritative. Treating the local copy as authoritative is how divergence bugs arise, so every row carries `last_synced_at` and reconciliation is a scheduled job. This is also what makes external-change detection (§5.7) coherent.
- **guardrail_config is versioned and immutable.** A change creates a new version rather than updating the row, because `optimization_action` references the version that authorised it. If guardrail rows were mutable, the audit trail would silently rewrite history.

9.2 Metrics store

Performance metrics have a different shape from OLTP data: high write volume, append-mostly, queried by time range and aggregate. Keeping them in the primary database unqualified would degrade both.

Phase	Store	Rationale
1–2	PostgreSQL + TimescaleDB hypertables	One database to operate; continuous aggregates give fast dashboards; adequate to low thousands of merchants
3+	BigQuery, with Postgres retaining recent hot data	Cross-merchant analytics, agency roll-ups, and Google's first-party Ads-to-BigQuery transfer

Granularity: hourly for the current day (anomaly detection needs intra-day resolution), daily beyond that, with 90-day and 13-month continuous aggregates maintained for dashboards and trailing-window rules.

Dashboards read only from this store, never from live GAQL. DRD §8.3 requires this, and the quota model in §5.4 makes it mandatory rather than merely advisable: a dashboard that issues GAQL per page load turns user traffic into quota consumption, which is the fastest route to RESOURCE_EXHAUSTED on a day when it matters.

9.3 Data retention and deletion

Data class	Retention	Deletion behaviour on merchant request
Account, profile, config	Account lifetime	Deleted or anonymised within 30 days
OAuth refresh tokens	Until revoked	Deleted immediately; Google-side grant revoked via API
Raw crawled HTML	30 days	Deleted immediately
Generated assets and prompts	24 months	Retained in de-identified form for model quality analysis only if the merchant consented
Performance metrics	25 months	De-identified and aggregated, or deleted on request
Optimization Action Log	7 years	Retained — legitimate interest in dispute and audit records; de-identified of personal data
Billing records	7 years	Retained — statutory requirement
Admin audit log	7 years	Retained
Application logs	30 days	Automatic

The one nuance worth surfacing to legal counsel: a deletion request cannot simply erase the Optimization Action Log, because it is the record of changes made to a third party's (Google's) system on the merchant's behalf and is the evidence in any billing or performance dispute. The approach is de-identification — strip personal data, retain the action record — and the privacy policy must state this clearly. See §12.4.

10 Reporting, Notifications and Internal Admin

10.1 Merchant dashboard

Element	Default view	Advanced view
Headline metrics	Spend, results (leads/calls/sales), cost per result, trend vs. prior period	Impressions, clicks, CTR, CPC, conv. rate, impression share
Time series	Results and spend, 30/60/90 day	Per-metric charts, per-campaign breakdown
Activity feed	Plain-English optimization log (“We paused 3 keywords that weren’t converting”)	Raw action records with before/after values
Campaign list	Plain-language names with status in words	Google entity names, IDs, bidding strategy, budget
Alerts	Plain-English issues with a clear next action	Raw policy topics and API error detail
Search terms	“What people searched to find you” (top converting)	Full search term report

The narrative strings are generated server-side (§7.1) and stored on the `optimization_action` and `notification_event` rows, so the dashboard, the email digest and the support console all show identical wording. Generating them per-surface would produce inconsistencies that undermine trust.

Advanced view must satisfy the RMF reporting requirements (§12.3) — account, campaign, ad group, ad and keyword level metrics including clicks, cost, impressions and conversions, with status. Designing the “simple” view without designing the RMF-compliant advanced view alongside it will fail the Standard Access review.

10.2 Weekly digest email

Generated as a scheduled batch job using the Batch API at 50% cost (§7.3). Content: what happened, what PPCWay did, what it means, and one recommended action if any. Deliberately short.

Failure rule: if the underlying metrics are stale or the account is in a broken state (suspension, billing hold, tracking failure), the digest reports **that** rather than reporting performance numbers that are misleading. A digest that says “spend \$0, results 0” when the real story is “your card was declined” actively damages trust.

10.3 Notifications

Event	Channel	Severity
Campaign approved and live	In-app + email	Info
Policy disapproval	In-app + email	Warning
Account suspension	In-app + email	Critical
Billing issue / card declined	In-app + email	Critical
Budget nearly exhausted	In-app + email	Warning
Significant performance change	In-app + email (digest unless severe)	Info/Warning
Change proposed, approval needed	In-app + email	Action required
Google connection lost	In-app + email	Critical
Conversion tracking stopped reporting	In-app + email	Critical
Subscription payment failed	In-app + email	Critical

Preference centre per DRD §5.8, with the constraint that Critical-severity notifications cannot be disabled — they concern money and account survival. Every notification carries a specific next action; a notification the merchant cannot act on is noise.

10.4 Internal admin console

Per DRD §5.9:

Capability	Detail
Cross-merchant health board	Link status, billing status, account status, tracking status, last successful optimization run
API error rates	By service, by error code, by merchant; trend and spike alerting
Quota dashboard	Operations used today vs. ceiling, by job class and service; projection to the daily boundary
Job / workflow status	Temporal workflow states, failures, retry counts, stuck workflows
Optimization queue	Pending approvals by merchant and age; expiring proposals
Manual override tools	Force re-sync, re-run generation, pause all automation for a merchant, revert an action
Impersonation / support view	Read-only merchant view, explicitly logged, with the merchant notified
Admin audit log	Every internal action with actor, target, before/after, and a required reason string

Two controls that matter: support impersonation is **read-only by default**, with write access requiring a second approver; and every admin action requires a free-text reason, which is the cheapest effective deterrent against casual use of override tools.

11 Required Accounts, Credentials and API Keys

This section is the operational checklist for standing the platform up. Items marked **BLOCKING** gate Phase 1 delivery and should be started in week 1, because several have external review times that cannot be compressed.

11.1 Google (largest and slowest set)

Ref	Item	Source	Lead time / cost	Notes
G1	Google Ads Manager Account (MCC)	Google Ads — Manager accounts	Same day; Free	BLOCKING. Must be a manager account, created under A1 Inc.'s corporate identity, not a personal Google account
G2	Google Cloud project	Google Cloud Console	Same day; Free tier	BLOCKING. Owns the OAuth client and is the project brand verification attaches to
G3	OAuth 2.0 client ID + secret (Web application)	GCP Console — Credentials	Same day; Free	BLOCKING. Redirect URIs must be registered per environment
G4	OAuth consent screen configured + verified	GCP Console	2–6 weeks if sensitive scopes require verification; Free	BLOCKING for public launch. adwords is a sensitive scope; unverified apps are capped at 100 users
G5	Google Ads API developer token	Google Ads — Tools — API Center (under the MCC)	Same day (Test), ~5 business days (Basic), ~10 business days (Standard); Free	BLOCKING. See §5.1. Apply for Basic immediately, Standard as soon as a demoable UI exists
G6	Google Cloud brand verification	GCP Console (project G2)	Days; Free	Strongly recommended — since July 2026 it can reduce developer-token review from days to hours
G7	Google Ads test account	Created under the MCC as a test account	Same day; Free	BLOCKING for CI. Integration tests must never touch a live merchant account
G8	Google Tag Manager API access	Enable in GCP project G2; same OAuth client	Same day; Free	Needed for automated conversion setup (DRD §5.3)
G9	Google Analytics Admin API access	Enable in GCP project G2	Same day; Free	For GA4 detection and conversion import
G10	PageSpeed Insights API key	GCP Console	Same day; Free tier, then per-call	Quality-Score-oriented page signals
G11	A1 Inc. Google payments profile	Google Payments	Days; n/a	Determines the account-creation weekly ceiling in §13.2 — monthly invoicing raises it substantially

The two items most often missed are G4 and G6. OAuth consent-screen verification for a sensitive scope is a separate Google review from the developer token, runs on its own timeline, and caps the app at 100 users until it completes. Discovering this in launch week has sunk launch dates on comparable products. Start both in week 1.

11.1.1 Google credential reference

Credential	Scope of secrecy	Where used	Rotation
Developer token	Platform-wide; one token for all merchants	ads-gateway only, as a request header	Only if compromised (invalidates nothing else, but Google must be notified)
OAuth client secret	Platform-wide, per environment	api (auth flow) and ads-gateway (token refresh)	Annually, or on staff departure with access
Per-merchant refresh token	Per merchant	ads-gateway only	On merchant re-consent; deleted on disconnect
login-customer-id header	Not secret	Every Google Ads call on a client account	n/a — set to the MCC ID

Note the risk concentration: **one developer token serves every merchant**. If it is suspended for policy violation, the entire platform stops for every merchant simultaneously. This is the highest-severity single point of failure in the product and it is why the policy validator (§7.4) and the guardrail layer (§5.5) are not optional engineering niceties. See risk R1 in §14.

11.2 Anthropic

Ref	Item	Source	Lead time / cost	Notes
A1	Anthropic Console organisation	Anthropic Console	Same day; Free to create	Set up under A1 Inc., not an individual
A2	API key (production)	Anthropic Console	Same day; Usage-based	BLOCKING . Store in Secret Manager; never in source or CI logs
A3	API key (staging/dev, separate workspace)	Anthropic Console	Same day; Usage-based	Separate workspace gives separate spend limits and rate limits
A4	Spend limit + usage alerts	Anthropic Console	Same day	Set a hard monthly cap well above the \$7.7 projection; alert at 50/80%
A5	Rate-limit tier review	Anthropic Console or support	Days	Confirm the tier supports peak onboarding concurrency before launch

Two practical notes. First, a single production key is sufficient; per-merchant keys are neither required nor useful, and would complicate caching. Second, keep the environments in **separate workspaces** rather than separate keys in one workspace, so a load test in staging cannot consume production rate limit.

11.3 Payments and email

Ref	Item	Source	Lead time / cost	Notes
S1	Stripe account	Stripe	Same day; full activation days; Per-transaction	BLOCKING . Canadian entity; confirm supported merchant currencies
S2	Stripe secret key + publishable key (live + test)	Stripe Dashboard	Same day	Test keys in staging only
S3	Stripe webhook signing secret	Stripe Dashboard	Same day	Signature verification is mandatory, not optional
S4	Products, prices, tax settings configured	Stripe Dashboard	Days	Starter/Growth/Agency tiers; Stripe Tax for CA/US
S5	Stripe Customer Portal configured	Stripe Dashboard	Same day	Satisfies self-serve plan management (DRD §5.7)
E1	Transactional email provider (Postmark, or A1's existing)	Postmark	Same day; Per-message	BLOCKING . Reuse A1's existing provider if there is one (DRD §7)
E2	Server API token	Provider dashboard	Same day	Separate tokens for transactional vs. digest streams

Ref	Item	Source	Lead time / cost	Notes
E3	SPF, DKIM, DMARC on the sending domain	A1's DNS provider	Hours, plus propagation; Free	BLOCKING for deliverability. Critical alerts landing in spam is a product failure
E4	Dedicated sending subdomain	A1's DNS provider	Hours; Free	e.g. mail.ppcway.com, to protect the root domain's reputation

11.4 Infrastructure and tooling

Ref	Item	Source	Lead time / cost	Notes
I1	Cloud provider account with billing	GCP or AWS	Same day; Usage-based	Separate projects/accounts per environment
I2	Temporal Cloud namespace	Temporal Cloud	Days; Usage-based	Self-hosting is possible but adds meaningful ops burden
I3	Domain + DNS for ppcway.com	A1's registrar	Existing; Existing	Subdomains: app., api., mail.
I4	TLS certificates	Managed by the load balancer / Cloud Run	Automatic; Free	
I5	GitHub organisation + private repos	GitHub	Same day; Per-seat	Actions minutes for CI
I6	Container registry	Artifact Registry or ECR	Same day; Usage-based	
I7	Sentry (or equivalent) project	Sentry	Same day; Per-event	Error tracking with release tagging
I8	Observability backend	Cloud Operations or Datadog	Same day; Usage-based	Dashboards per §15.1
I9	Uptime monitoring + on-call paging	Better Stack or PagerDuty	Same day; Per-seat	Critical alerts must page a human
I10	Log retention / SIEM (if required)	Cloud Logging	Same day; Usage-based	30-day application logs, longer for audit logs
I11	Stock image library (optional)	Getty or Shutterstock	Days; Subscription	PMax image fallback (§6.7)

11.5 Secret management design

No secret is stored in source control, in a container image, in an environment file committed to a repository, or in a CI log. The pattern:

```

Google Secret Manager (or AWS Secrets Manager)
|
|-- holds: developer token, OAuth client secret, Anthropic key,
|         Stripe keys, Postmark token, DB credentials, Temporal
|         API key, Sentry DSN
|
+-- accessed at runtime by service identity (workload identity),
    never by a long-lived service-account key file

Cloud KMS
|
+-- wraps per-merchant data encryption keys used for OAuth refresh
    tokens (envelope encryption -- §12.1)

Per-merchant OAuth refresh tokens
|
+-- ciphertext in PostgreSQL; data key wrapped by KMS;
    decrypted only inside ads-gateway, only in memory,
    never logged, redacted from all error reporting

```

Rotation schedule: Anthropic and Stripe keys quarterly; OAuth client secret annually and on relevant staff departure; database credentials quarterly via automated rotation; the Google Ads developer token only on compromise. Every rotation is a documented runbook, tested at least once before it is needed in anger.

11.6 Environment variable reference

Full list in Appendix C. The set that must exist before the application starts:

GOOGLE_ADS_DEVELOPER_TOKEN	secret
GOOGLE_ADS_CLIENT_ID	secret
GOOGLE_ADS_CLIENT_SECRET	secret
GOOGLE_ADS_LOGIN_CUSTOMER_ID	config (the MCC ID, digits only)
GOOGLE_ADS_API_VERSION	config (pinned, e.g. v25 -- see S5.8)
GOOGLE_OAUTH_REDIRECT_URI	config (per environment)
GTM_API_ENABLED	config
PAGESPEED_API_KEY	secret
ANTHROPIC_API_KEY	secret
LLM_MODEL_COPY	config (e.g. claude-opus-5)
LLM_MODEL_NARRATIVE	config
LLM_PROMPT_VERSION	config (recorded on every generated asset)
LLM_MONTHLY_TOKEN_BUDGET	config (cost circuit breaker)
STRIPE_SECRET_KEY	secret
STRIPE_WEBHOOK_SECRET	secret
POSTMARK_SERVER_TOKEN	secret
DATABASE_URL	secret
REDIS_URL	secret
TEMPORAL_ADDRESS	config
TEMPORAL_NAMESPACE	config
TEMPORAL_API_KEY	secret
KMS_KEY_RING	config
KMS_TOKEN_KEY	config
SENTRY_DSN	secret
ENVIRONMENT	config (dev staging production)

A startup check validates that every required variable is present and that every secret resolves, and refuses to start otherwise. A service that boots with a missing Google credential and fails only when the first merchant onboards is far more expensive to diagnose.

12 Security and Compliance

12.1 Credential and token protection

DRD §8.1 requires OAuth tokens and API credentials encrypted at rest and never logged in plaintext. Implementation:

Control	Implementation
Envelope encryption	Per-merchant data encryption key (DEK) generated per link; DEK wrapped by a Cloud KMS key; only the wrapped DEK and ciphertext are stored
Decryption boundary	Refresh tokens are decrypted only inside ads-gateway, only in process memory, never written to disk or to a log
Log redaction	A structured-logging filter drops any field named in a denylist (token, secret, refresh_token, authorization, client_secret) and pattern-matches token shapes as a backstop
Error reporting	Sentry before_send scrubs request bodies and headers on any ads-gateway frame
Transport	TLS 1.2+ everywhere; internal service-to-service traffic authenticated by service identity

Control	Implementation
At rest	Cloud SQL and Cloud Storage encryption enabled; backups encrypted
No long-lived key files	Workload identity for service-to-service auth; no service-account JSON keys in images or CI
Secret access audit	Secret Manager access logs retained and alerted on anomalous access patterns

12.2 Authentication and authorization

Use a managed identity provider. Password storage, reset flows, TOTP enrolment, session management, breach detection and account-takeover defence are a large, unglamorous, high-consequence surface. Auth0, Clerk or Firebase Auth all satisfy DRD §5.1 including TOTP MFA and Google SSO, and none of that logic should be written for this project.

Role matrix (DRD §5.1):

Capability	Owner	Team Member	Agency Viewer	Internal Admin
View dashboard and reports	Yes	Yes	Yes	Yes (logged)
Edit business profile	Yes	Yes	No	No
Approve campaign launch	Yes	Yes	No	No
Approve optimization proposals	Yes	Yes	No	No
Change autonomy level	Yes	No	No	No
Change guardrail limits	Yes	No	No	Yes (logged, reason required)
Connect / disconnect Google Ads	Yes	No	No	No
Manage billing and subscription	Yes	No	No	No
Invite / remove users	Yes	No	No	No
Export data / request deletion	Yes	No	No	On request, logged
Cross-merchant admin tools	No	No	No	Yes (logged, reason required)

MFA is available to all roles and **required** for Internal Admin. Sessions are short-lived with refresh; a password or MFA change invalidates all sessions.

12.3 Google Ads API policy compliance and RMF

Two distinct obligations, and it is worth being precise because they are frequently conflated.

Google Ads API Terms & Conditions and API policies. The obligations that bear on design:

Obligation	How PPCWay satisfies it
No sharing of the developer token or API access with unrelated third parties	Single token, held only in ads-gateway; no reseller or white-label arrangement exposes API access directly (Phase 3 white-label is a UI skin over PPCWay's own token)
User control and transparency over automated changes	Autonomy levels (§8.3), explicit Full-Auto consent record, plain-English activity feed, revert capability
Accurate representation to users	Plain-English narratives are generated from actual action records, not marketing claims
Compliance with Google Ads advertising policies	Policy validator (§7.4) plus <code>validate_only</code> pre-flight; restricted-category gate before generation

Obligation	How PPCWay satisfies it
Respect for API rate limits and quotas	Quota accounting and backoff (§5.4, §5.6)
Data-use restrictions on Google Ads data	Google Ads data is used only to serve that merchant; cross-merchant use is limited to aggregated, de-identified category benchmarks, and only where terms permit — confirm this specific use with Google before building it

Required Minimum Functionality (RMF). RMF applies **only to Standard Access** tokens, and PPCWay is a Full-Service Tool, so all three categories apply. This directly constrains v1 UI scope, and it is the item most likely to be discovered late:

RMF category	What PPCWay must expose to merchants
Creation	Campaign creation with geo and language targeting; conversion tracking setup; ad group and keyword creation; bidding strategy selection (Target CPA / Target ROAS / Maximize Conversions); budget configuration; extensions/assets where applicable
Management	Edit campaign settings and budgets; pause / enable / remove campaigns, ads and keywords; modify bidding strategy; campaign scheduling
Reporting	Metrics at customer, campaign, ad group and keyword level — clicks, cost, impressions, conversions — plus status for paused/active/removed, search term reporting, and bidding strategy reporting

This is in direct tension with the product's simplification goal, and the tension must be resolved deliberately rather than discovered in review. DRD §5.6 wants no Google Ads jargon by default; RMF requires a specific set of controls and reports to exist. The resolution is the **Advanced view**: the default view stays plain-English, and Advanced view is where RMF compliance lives. Advanced view is therefore **not** a Phase 2 nice-to-have — it is a Phase 1 requirement for obtaining Standard Access. Budget it accordingly (§16).

Google reviews the actual tool. Request the current RMF feature checklist from the Google Ads API compliance team before the Standard Access application, and walk the UI against it.

12.4 Data privacy: PIPEDA and GDPR

A1 Inc. is Ontario-based, so PIPEDA applies; DRD §8.2 also asks for GDPR-equivalent flows if EU merchants are targeted.

Requirement	Implementation
Lawful basis / consent	Explicit consent at signup for processing; separate explicit consent for Full-Auto automation
Purpose limitation	Data collected for onboarding and optimization is used only for that; secondary use requires separate consent
Access / portability	Self-serve export of profile, campaign metadata, action log and metrics as CSV/JSON
Correction	Editable profile; corrections propagate to future generation runs
Deletion	Self-serve deletion request; 30-day completion; per the retention matrix in §9.3
Data minimisation	Crawled raw HTML deleted after 30 days; no collection of merchants' end-customer personal data
Breach notification	Documented incident-response process with regulator and merchant notification paths
Sub-processors	Register maintained and disclosed: Google, Anthropic, Stripe, Postmark, cloud provider, Temporal, Sentry
Cross-border transfer	Disclosed; if EU merchants are onboarded, EU data residency or SCCs must be assessed before launch in that market

Two points requiring legal sign-off rather than an engineering decision:

1. **Merchant website content is crawled and sent to a third-party LLM provider.** This must be disclosed at the point of crawl, and the sub-processor list must name Anthropic. Merchants whose sites contain personal data (staff bios, testimonials) are transmitting it.
2. **The Optimization Action Log is retained for 7 years and is not fully deletable (§9.3).** The privacy policy must state the retention basis explicitly.

12.5 PCI scope

Money flow	Who handles the card	PPCWay's PCI scope
PPCWay SaaS subscription	Stripe (Checkout / Elements)	SAQ-A: card data never touches PPCWay servers
Google Ads media spend	Google, on Google's own billing pages	None — PPCWay never sees or transmits the card

This is a genuine benefit of the DRD §12 decision not to take custody of ad spend, and it is worth protecting. If A1 Inc. later decides to intermediate ad spend, the change is not merely commercial: it introduces money-transmission licensing questions and materially expands PCI and financial-controls scope. That should be treated as a separate programme, not a feature.

12.6 Crawler isolation and SSRF defence

The crawler fetches attacker-influenced URLs, so it is treated as a hostile-input boundary:

Control	Implementation
Network isolation	Separate Cloud Run service with its own service account holding no platform secrets
Egress policy	Deny by default; allow only outbound HTTP/HTTPS on 80/443 to public addresses
SSRF blocking	Resolve DNS first and reject private, loopback, link-local and metadata addresses (127.0.0.0/8, 10/8, 172.16/12, 192.168/16, 169.254/16, ::1, fd00::/8); re-validate after every redirect
Redirect limits	Max 5 redirects; scheme restricted to http/https; reject file:, gopher:, ftp:
Resource limits	Response size cap, total time budget, CPU and memory limits on the container
Content handling	Parsed as untrusted data; no evaluation of page JavaScript outside the sandboxed headless browser; no shell interpolation of any extracted string
Prompt-injection posture	Extracted text is delimited and labelled untrusted in prompts; the generation call has no tools and no credentials; output must pass the validator regardless (§7.5)

The 169.254.169.254 metadata endpoint deserves specific mention: a merchant-supplied URL that redirects there, on a crawler with an over-privileged service account, is a direct route to cloud credentials. Both controls — address blocking *and* a credential-free service account — are required; neither alone is sufficient.

12.7 Dependency and supply-chain hygiene

Control	Implementation
Dependency scanning	Dependabot / Renovate with automated PRs; CI fails on High/Critical
Container scanning	Image scanning on push to Artifact Registry
Lockfiles	Committed and enforced; reproducible builds
SAST	CodeQL or equivalent in CI
Secret scanning	Pre-commit hooks plus GitHub secret scanning with push protection
Least-privilege CI	Short-lived OIDC federation to the cloud provider; no static deploy keys
Signed images	Provenance attestation on release images

Control	Implementation
Pinned base images	Digest-pinned, updated on a schedule rather than implicitly

13 Limitations and Constraints

Everything in this section is a property of the platforms PPCWay depends on, or a deliberate scope decision. None of it is removable by better engineering, and all of it should be understood before commitments are made to customers or investors.

13.1 The billing hand-off cannot be eliminated

Constraint. Google does not permit programmatic entry of payment-card data, and a Google payments profile can only be created through the Google Ads UI. The API can *read* billing state (*billing_setup*, serving eligibility) but cannot create or complete it.

Consequence. The onboarding funnel has a mandatory external step. A merchant with no existing Google Ads payment method must leave PPCWay, complete Google’s billing flow, and return. This directly threatens the under-15-minute target in DRD §2 for exactly the segment the product targets — merchants who have never advertised before.

Mitigations (all partial).

- Detect billing state immediately after linking and warn at minute 1, not minute 12.
- Deep-link directly into the correct Google billing page with the customer ID pre-scoped.
- Make the campaign fully approvable while billing is outstanding, so returning is one click.
- Poll billing state and email the merchant the moment it clears, with a launch link.
- Measure and report “time to live campaign, excluding time in Google billing” separately from wall-clock, so the funnel metric is actionable rather than dominated by an external step.
- Report the 15-minute target as **median time-in-PPCWay**, and state the caveat publicly rather than being contradicted by merchant experience.

13.2 Account provisioning has a hard weekly ceiling

Constraint. New Google Ads account creation is limited by the payments profile type of the creating manager account: approximately **15 accounts per week** on an automatic or manual payments profile, and up to **1,500 per week** with a monthly invoicing profile. Additionally, since March 2025 *CreateCustomerClient* returns errors when the manager account is below Google’s spending thresholds or has a policy issue — so a new MCC may not be able to create accounts at all initially.

Consequence. The Phase 2 “we’ll create your Google Ads account for you” flow — a significant differentiator — is throughput-capped at roughly two accounts per day until A1 Inc. qualifies for monthly invoicing with Google.

Mitigations.

- Treat monthly-invoicing qualification as a **commercial prerequisite for Phase 2**, and open that conversation with Google early; it depends on spend history and account standing.
- Keep Path A (link an existing account) as the primary flow, and treat Path B as an assisted option with a queue and an honest expectation set.
- Instrument the creation-error codes specifically, so a threshold or policy rejection is distinguishable from a transient failure.
- Confirm current figures with Google before committing a Phase 2 date. Google publishes overlapping weekly ceilings in different places; the payments-profile-dependent limits are the operative ones, but the exact numbers must be verified at build time.

13.3 API quota ceilings bound merchant count

Tier	Merchants supportable at ~40 ops/day	Notes
Explorer	~70, and blocks required services	Unusable
Basic	~300–375	Closed beta only
Standard	thousands+	Bounded by per-customer-ID QPS, not a daily cap

Even on Standard Access, the per-customer-ID limits (§5.4) mean **generation latency for a single merchant cannot be reduced by adding workers**. Keyword planning is capped near 1 QPS per customer ID. The platform scales horizontally across merchants and is essentially serial within one. Onboarding concurrency at peak must be sized accordingly.

13.4 The API version treadmill is permanent

Four major versions a year, each supported for twelve months. There is no configuration in which PPCWay stops doing migration work. Budget 2–4 engineer-days per quarter indefinitely, with occasional larger spikes. Any project plan or cost model that omits this line is wrong.

13.5 Conversion tracking is the single point of failure

Constraint. Every claim in the product thesis — optimize CPA, optimize ROAS, prove value — depends on conversion data that PPCWay does not control. The tag lives on the merchant’s site. It breaks when they redesign, change platforms, adjust a consent banner, or a plugin updates.

Consequence. Without conversion data the Optimization Engine cannot compute CPA or ROAS and degrades to optimizing clicks, which is close to worthless for a lead-generation SMB. Worse, *silently broken* tracking means the engine optimizes toward zero-conversion signals and can actively damage an account.

Mitigations.

- Treat conversion verification as a **blocking gate** before launch: a real, observed test conversion, not merely a tag that exists (§6.5 refuses to launch without it).
- Continuously monitor for conversion collapse (§8.5) and treat it as a tracking fault, not a performance fault, suppressing bid changes while it persists.
- Prefer GTM API auto-configuration over hand-pasted snippets, which are the most fragile option.
- Where GA4 exists, import GA4 conversions as a secondary signal for cross-checking.
- Be explicit in the merchant UI when optimization is degraded due to missing conversion data.

Additional constraints worth stating plainly: consent-mode and privacy changes suppress a proportion of conversions in some regions, meaning reported conversions are systematically lower than actual; and Google’s attribution and conversion modelling means numbers will not reconcile exactly with a merchant’s CRM. Support will field this question regularly, so the UI should pre-empt it.

13.6 Google Ads platform entity limits

Not binding for a typical SMB, but they constrain generation logic and the agency tier:

Limit	Value
Non-manager accounts linked to one MCC	85,000
Campaigns per account (active + paused)	10,000
Ad groups per campaign	20,000 (Local and App: 100)
Enabled responsive search ads per ad group	3

Limit	Value
Ads per account (active + paused)	4 million
Negative keywords per campaign	10,000
Negative keyword lists per manager or child account	20
Ad group targeting items per account	5 million
Campaign targeting items per account	1 million
Location targets per campaign	10,500
PMax asset group: headlines / long headlines / descriptions	15 / 5 / 5
PMax asset group: images / logos / videos	20 / 5 / 15

The negative keyword **list** limit of 20 is the one that actually bites: a naive design that creates a shared list per merchant per category exhausts it quickly. Lists must be composed at the platform level and shared (§6.6).

13.7 What the Optimization Engine cannot do

Managing expectations here is a product-integrity issue, because the marketing promise (“continuously improve CPA, ROAS, CTR”) is easy to over-read.

Cannot	Why
Fix a bad offer or an uncompetitive price	No amount of bidding optimization overcomes weak product-market fit
Fix a poor landing page	PPCWay can flag page-speed and relevance signals; it cannot rewrite the merchant's site
Overcome insufficient budget	Below roughly \$300–500/month in competitive verticals there is not enough data for the engine to learn from
Beat Google's own automated bidding at bidding	Smart Bidding sees auction signals no API caller can access. PPCWay's value is structure, hygiene, negatives, creative and pacing — not out-bidding the auction
Reverse spend already incurred	Structural changes revert; money does not
Rewind Google's learning phase	A strategy change resets learning; reverting resets it again
Guarantee policy approval	The validator reduces disapprovals; Google's decision is final and can change retroactively
Optimize on a broken conversion signal	See §13.5
Act instantly on a disapproval	Detection is by polling; expect up to a 12-hour delay at the recommended cadence
Produce results in the first 2–4 weeks	Learning periods and data accumulation mean early results are unrepresentative. The product must set this expectation at onboarding or churn in month 1 will be severe

13.8 LLM limitations

Limitation	Practical effect	Mitigation
Non-determinism	The same profile can produce different copy on two runs	Prompt and model versions recorded per asset; golden-set evaluation gates changes
Fabrication	Can invent services, offers or credentials	Landing-page substantiation check (§7.4)
No ground truth on policy	Cannot reliably self-assess Google policy compliance	Deterministic validator plus Google <code>validate_only</code>
Character-limit unreliability	Counts characters imperfectly	Enforced after generation, not trusted in generation
Latency variability	Occasional multi-second outliers	Temporal activity timeouts; progress UI; partial results

Limitation	Practical effect	Mitigation
Refusals	Occasional refusal on a benign request	Detect <code>stop_reason == "refusal"</code> ; template fallback
Provider dependency	Outage or price change affects unit economics	Thin abstraction (§7.6); template fallback; monitored spend caps
Quality drift on model change	A newer model can change tone or length	Golden-set evaluation before any model change ships
Prompt injection via crawled content	Attacker-controlled text enters the prompt	No tools or credentials on the generation call; output validated regardless

13.9 Commercial and policy limitations

- **Restricted verticals are excluded at launch** (healthcare, financial services, gambling, adult, political, and others). Serving them requires Google certifications, per-vertical policy logic, and in some cases local licensing. This removes a meaningful share of the SMB market from the addressable set (Assumption A4).
- **One developer token serves all merchants.** A policy suspension halts the entire platform. Risk R1, §14.
- **PPCWay does not control ad spend billing**, so it cannot stop a merchant's spend by declining a payment. If a subscription lapses, campaigns keep running under the merchant's own Google billing. Whether PPCWay pauses campaigns on subscription lapse is a policy decision with contractual implications and needs an explicit answer before launch.
- **Google can change anything** — policy, limits, entity models, campaign types. Performance Max itself did not exist a few years ago. The rules engine and validator must be configuration-driven so that a Google change is a config update, not a redeploy.

13.10 Explicitly out of scope for v1

Restating DRD §12 with the engineering consequences:

Out of scope	Note
PPCWay taking custody of ad spend	Would introduce money-transmission and PCI scope (§12.5)
Non-Google channels (Meta, Microsoft, TikTok)	Each is a separate integration of comparable size to the Google one
Fully unsupervised Full-Auto at launch	Phase 1 ships Approve-First only (DRD §11)
App campaigns	Separate asset pipeline and conversion model
Shopping feed management	Merchant Center integration is its own project; PMax with a feed assumes the feed already exists
AI image generation for creative	Licensing and brand-accuracy risk outweighs benefit at MVP (§6.7)
Multi-language ad copy	Adds locale-specific policy rules and native-quality review (Assumption A7)
Call tracking / recording	Separate telephony integration and its own consent regime

14 Risk Register

Severity is the product of likelihood and impact on the business, not just on the codebase. Unless noted otherwise the owner is the engineering lead; **R2, R5 and R10 are owned by the product owner**, because their mitigations are commercial and product decisions rather than engineering ones.

ID	Risk	Sev	Mitigation
R1	Developer token suspended for policy violation — entire platform stops for every merchant	Critical	Policy validator + <code>validate_only</code> pre-flight; guardrails; RMF compliance; monitor disapproval rates as a leading indicator; maintain a direct Google API compliance contact; documented incident plan
R2	Standard Access not granted before launch	Critical	Apply in Phase 0; complete brand verification; build Advanced view early for RMF; keep a beta-scale launch plan as fallback
R3	Automated change causes runaway spend	Critical	Guardrail layer with hard caps, cumulative delta limits, blast-radius cap, fail-closed (§5.5); Approve-First default in Phase 1; anomaly detection
R4	Broken conversion tracking causes the engine to degrade accounts	High	Blocking verification gate; conversion-collapse detection suppresses bid changes; explicit degraded-mode UI
R5	OAuth consent-screen verification delays public launch	High	Start in week 1; scope minimisation; 100-user cap is workable for closed beta
R6	Generated ad copy makes a false claim, causing a merchant complaint or legal exposure	High	Landing-page substantiation; prohibited-claim rules; merchant approval gate on first launch; clear terms on merchant responsibility for approved copy
R7	Google Ads API breaking change lands mid-sprint	Medium	Pinned versions; quarterly upgrade slot; contract tests against the live test account; never within 90 days of sunset
R8	Quota exhaustion during a growth spike	Medium	Priority classes and load shedding (§5.4); alerting at 60/80/92%; Standard Access
R9	Merchant revokes OAuth, leaving unmanageable live campaigns	Medium	Revocation detection; billing and UI state change; re-connect flow
R10	Churn in month 1 because results lag expectations	High	Set expectations at onboarding; report learning-period status honestly; weekly digest explains what is happening and why
R11	Temporal learning curve slows Phase 1	Medium	One week of team ramp-up budgeted; Celery fallback documented (§3.3)
R12	LLM cost or quality regression from a prompt change	Low	Golden-set evaluation gate; token budget caps; cache-hit monitoring
R13	Crawler SSRF or prompt injection via a merchant site	Medium	Isolated credential-free service; address blocking; no tools on generation calls (§12.6)
R14	Cross-tenant data exposure	Critical	Row-level security; guardrail scope check; integration tests that assert isolation
R15	Google withdraws or changes a campaign type PPCWay depends on	Low	Configuration-driven rules engine; campaign-type support is data, not code

15 Non-Functional Requirements and Service Objectives

Category	Objective	Measurement
Availability	99.5% monthly for the merchant app; 99.9% for the optimization pipeline (it is asynchronous and can absorb short outages)	Uptime monitoring; Temporal workflow success rate
Dashboard latency	p95 < 800 ms for the default view; p99 < 2 s	Served from pre-aggregated store (§9.2)
API latency	p95 < 300 ms for reads; p95 < 1 s for writes	OpenTelemetry traces
Onboarding	Median time-in-PPCWay < 15 min, excluding time in Google's billing flow (§13.1)	Funnel instrumentation with an explicit external-wait segment
Generation	p95 campaign generation < 3 min from wizard completion	Temporal workflow duration
Optimization freshness	Metrics no more than 6 hours stale for rule evaluation; guardrail check 9 enforces this	Ingestion lag metric
Mutation correctness	Zero unlogged mutations; zero mutations bypassing guardrails	Reconciliation job; audit assertion in CI
Data durability	RPO 15 min, RTO 4 h	Point-in-time recovery on Cloud SQL; restore drill quarterly
Scalability	5,000 merchants without re-architecture	Load test at 2x projected peak before each phase ships

Category	Objective	Measurement
Quota headroom	Never exceed 80% of the daily ceiling in normal operation	Quota ledger and alerting
Security	No High/Critical vulnerability older than 7 days in production	Dependency and image scanning

15.1 Observability

Three dashboards are required from day one, because they are what makes the platform operable:

1. **Google Ads API health** — operations consumed vs. ceiling (with projection to the daily boundary), error rate by service and error code, retry and backoff rates, per-customer-ID throttle events. Required explicitly by DRD §§5.9 and 6.8.
2. **Optimization pipeline health** — workflow success/failure by job type, queue depth and age, stuck workflows, actions by status, pending-approval age distribution, reconciliation discrepancies.
3. **Merchant health** — accounts by link/billing/account/tracking status, accounts with degraded optimization, accounts with no successful run in 48 h.

Alerting policy: page a human for anything that risks merchant money or account survival (quota exhaustion imminent, mutation failures spiking, suspension detected, spend anomaly, guardrail rejections spiking). Everything else goes to a queue reviewed on the next working day. An alerting policy that pages for non-actionable events trains the team to ignore pages, which is worse than having no alerts.

16 Delivery Plan and Effort Estimates

Estimates are in engineer-days for a mid-to-senior engineer familiar with the stack but new to the Google Ads API. They include tests and code review, and exclude product design, QA cycles and project management.

16.1 Phase 0 — Foundations and credentials (3–4 weeks elapsed)

Mostly gated by external review times, so it runs in parallel with early Phase 1 work.

Work item	Days
MCC, GCP project, OAuth client, developer token application, brand verification	3
OAuth consent screen submission and iteration	2
Terraform for three environments; Cloud Run, Cloud SQL, Redis, Secret Manager, KMS	6
CI/CD pipelines, image build, migrations, preview environments	5
Repo scaffolding: FastAPI + Next.js skeletons, lint/format/type gates	3
Temporal Cloud namespace, worker skeleton, local dev harness	3
Observability baseline: OTel, Sentry, dashboards, alert routing	4
Google Ads test account and first authenticated call end-to-end	2
Subtotal	28

16.2 Phase 1 — MVP (14–18 weeks elapsed)

Work item	Days
Auth integration (IdP), roles, RLS, session handling	8
Merchant/user/membership model, invitations, account settings	6
Stripe: Checkout, subscriptions, webhooks, Customer Portal, plan gating	9
ads-gateway: client wiring, token refresh, quota buckets, backoff, idempotency, <code>validate_only</code> , <code>partial_failure</code> handling	18
OAuth linking flow (Path A), invitation/accept, billing-state detection, revocation handling	10

Work item	Days
Onboarding wizard UI (multi-step, resumable, mobile-first)	14
Crawler service: fetch, parse, SSRF defences, PageSpeed signals, fallbacks	11
Keyword theme generation: <code>KeywordPlanIdeaService</code> , deterministic filters, LLM grouping, validation	10
LLM integration: prompts, caching, structured outputs, refusal handling, versioning	9
Policy validator (all checks in §7.4) with unit tests	11
Structure + bidding rules engine with table-driven tests	8
Campaign assembly and launch: Search campaigns, budgets, RSAs, negatives, conversion actions	14
Plain-English preview and approval UI	9
Conversion tracking assistant: GTM API, GA4 detection, verification gate	12
Metrics ingestion: GAQL pulls, TimescaleDB schema, continuous aggregates	10
Guardrail layer with full test coverage	10
Optimization Action Log, append-only enforcement, revert endpoint	8
Optimization jobs (Approve-First): pacing, search-term mining, anomaly detection, policy poll	16
Approval workflow: Temporal signals, expiry timers, UI	7
Merchant dashboard — default view	12
Advanced view (RMF compliance — not optional, see §12.3)	11
Notifications: in-app, email, preference centre	8
Internal admin console (Phase 1 subset)	10
Restricted-category gate	4
Data export and deletion flows	5
Integration test suite against the Google test account	10
Load test, security review, launch hardening	8
Subtotal	278

At 2 backend + 1 frontend engineers (roughly 15 productive engineer-days per calendar week), 278 days is approximately **18 calendar weeks**. Fourteen weeks is achievable only by deferring the internal admin console beyond its minimum, trimming notification scope, and accepting a thinner test suite — all of which are real trade-offs rather than free savings.

16.3 Phase 2 — Automation depth (10–14 weeks)

Work item	Days
Account provisioning (Path B): <code>CreateCustomerClient</code> , error handling, throughput queue	11
Performance Max: asset groups, image sourcing and validation, listing groups	20
Full-Auto mode: consent capture, expanded guardrails, staged rollout	10
Weekly digest: Batch API generation, templates, deliverability	9
Expanded reporting: PDF/CSV export, custom date ranges	11
Bidding-strategy transition automation and learning-period watchdog	8
RSA and PMax asset performance review and rotation	10
Anomaly detection expansion	8
Admin console expansion	7
Standard Access application support and RMF gap closure	5
Subtotal	99

16.4 Phase 3 — Scale and agency (12–16 weeks)

Work item	Days
Agency console: multi-client switching, consolidated reporting, permissions	25

Work item	Days
White-label: theming, custom domains, branded email	18
BigQuery migration for the metrics store	12
Advanced anomaly detection with seasonality	12
Sub-MCC sharding capability	8
Microsoft Ads evaluation spike (evaluation only, per DRD §11)	8
Subtotal	83

16.5 Critical path and sequencing

```

WEEK  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18
|--PHASE 0 (parallel)--|
[dev token: Basic ~1wk -> Standard application ~wk6 -> grant ~wk8]
[OAuth consent verification: submit wk1 -> grant wk3-7 <-- RISK]
|----- ads-gateway (18d) -----|          <-- CRITICAL PATH
|--- OAuth linking ---|
|----- onboarding wizard (frontend, parallel) -----|
|-- crawler --|
|   keywords + LLM + validator --|
|   campaign assembly ---|
|       conv. tracking --|
|       metrics ingest --|
|       guardrails --|
|       opt jobs --|
|           dashboards + advanced -|
|               admin -|
|               -hardening-|

```

The critical path is ads-gateway then campaign assembly then guardrails then optimization jobs. Frontend work parallelises well. **The two genuine schedule risks are external:** the Standard Access grant and OAuth consent-screen verification. Neither can be accelerated by adding engineers, which is why both must be started in week 1.

16.6 Team shape

Role	Phase 0	Phase 1	Phase 2	Phase 3
Backend engineer	1	2	2	2
Frontend engineer	0	1	1	1.5
DevOps / platform	1	0.5	0.25	0.25
Product owner (A1)	0.5	0.5	0.5	0.5
PPC subject-matter expert	0.25	0.5	0.5	0.25
QA	0	0.5	0.5	0.5

The **PPC subject-matter expert is not optional**. The negative keyword lists, the structure decision table, the bidding thresholds and the validator's prohibited-claim rules all encode domain expertise that engineers cannot supply. A plausible-looking rules engine written without that input is the most likely cause of a quietly underperforming product.

17 Running Cost Model

Monthly, USD, excluding salaries and excluding merchants' Google Ads media spend (billed directly by Google to the merchant).

Component	100 merchants	500 merchants	2,000 merchants
Cloud Run (api, web, workers)	90	260	750
Cloud SQL (PostgreSQL + Timescale, HA)	180	380	900
Redis (Memorystore)	45	90	180
Cloud Storage + egress	15	45	140
Temporal Cloud	100	250	700
Anthropic API (§7.7)	32	148	560
Google Ads API	0	0	0
Stripe (2.9% + 30c, assuming \$99 ARPU)	320	1,600	6,400
Postmark	15	50	180
Sentry	26	80	200
Observability	40	120	350
Uptime + paging	25	25	50
Identity provider	25	120	400
Stock imagery (optional)	50	50	100
Total	~963	~3,218	~10,910
Per merchant per month	~\$9.63	~\$6.44	~\$5.46
Excluding Stripe fees	~\$6.43	~\$3.24	~\$2.26

Observations:

- **The Google Ads API is free.** Its cost is engineering time (§13.4), not fees.
- **Stripe fees dominate** and scale linearly with revenue rather than with infrastructure. The “excluding Stripe” row is the more useful engineering figure.
- **LLM cost is not a concern at any of these scales** — roughly 3–5% of non-Stripe infrastructure cost. This is the quantitative basis for recommending the strongest model (§7.2) rather than the cheapest.
- **Cloud SQL is the largest controllable line item** at low volume. A single-zone instance in early beta roughly halves it, at the cost of the availability objective in §15.
- Fixed costs (paging, Sentry floor, stock imagery) make per-merchant economics poor below ~100 merchants. This is normal and improves quickly.

18 Environments, CI/CD and Testing

18.1 Environments

Environment	Google Ads	Stripe	LLM	Data
Local	Test account, mocked by default	Test keys	Real key, low-effort settings, tiny token cap	Synthetic
Staging	Test account only	Test keys	Separate workspace key	Anonymised or synthetic
Production	Live accounts under the MCC	Live keys	Production key	Real

Non-negotiable rule: staging must never hold credentials that can mutate a live merchant account. DRD §6.8 asks for a test-account environment for integration testing; this document strengthens that to a hard guarantee, enforced by which secrets exist in the staging project — the live developer token and live merchant refresh tokens are simply absent from it. A guarantee enforced by code review alone will eventually fail.

18.2 CI/CD

Stage	Gate
Pre-commit	Format, lint, secret scan
PR	Type check (mypy strict, tsc), unit tests, SAST, dependency audit, migration dry-run
PR	Audit assertion: no mutate call outside ads-gateway (static check on the Google Ads client import)
PR	Preview environment deploy for frontend review
Merge to main	Integration tests against the Google test account; contract tests against pinned API version
Deploy staging	Automatic; smoke tests; golden-set LLM evaluation
Deploy production	Manual approval; canary; automatic rollback on error-rate breach
Post-deploy	Release tagged in Sentry; dashboards checked

The static check that no module outside `ads-gateway` imports the Google Ads client is a small piece of CI that preserves the single most important architectural invariant in the system (§4.1). Architectural rules that are not enforced mechanically decay.

18.3 Testing strategy

Layer	Coverage
Unit	Rules engine (exhaustive table-driven), policy validator (one case per rule plus adversarial inputs), guardrails (one case per check plus fail-closed paths)
Contract	Recorded Google Ads API fixtures per pinned version; re-recorded on upgrade to detect breaking changes
Integration	Full generate-and-launch cycle against the Google test account, including <code>validate_only</code> and <code>partial_failure</code> paths
Workflow	Temporal replay tests to guarantee determinism; crash-recovery tests asserting no duplicate mutation
Isolation	Automated cross-tenant tests asserting merchant A cannot read or mutate merchant B
LLM	Golden-set evaluation: fixed profiles, assert validator pass rate, character compliance, no fabrication against the source page, and tone consistency
E2E	Playwright over signup, onboarding, preview, approval and dashboard
Load	2x projected peak onboarding concurrency and dashboard traffic
Chaos	Kill workers mid-mutation; assert reconciliation resolves every pending action correctly
Security	Dependency and image scanning in CI; annual penetration test

Two of these carry unusual weight for this product. The **chaos test** (“kill a worker between log and mutate, assert reconciliation gets it right”) directly validates the guarantee that DRD §8.4 asks for and that everything else in the audit design rests on. The **LLM golden-set evaluation** is the only mechanism that will catch a quality regression from a prompt or model change before merchants see it.

19 Answers to the DRD’s Open Questions

DRD §13 poses five questions for the development team. Each is answered here with the evidence and the decision it implies.

19.1 Q1: Confirm target Google Ads API access tier and current quota limits

Answer: Standard Access is required for launch. Basic Access supports a closed beta only. Explorer Access cannot run the product at all.

Verified figures as of August 2026: Explorer 2,880 operations/day against production accounts; Basic 15,000/day; Standard unlimited (subject to system rate limits). Explorer additionally blocks account creation, the planning services (including `KeywordPlanIdeaService`) and the billing services — three capabilities the product requires.

At an estimated ~40 operations per merchant per day (§5.4), Basic Access supports roughly 300–375 merchants in total across the whole platform. Standard Access is therefore a launch gate, not an optimization.

Architectural implication: the Optimization Engine must be quota-aware from day one, with priority classes and load shedding (§5.4), because even Standard Access retains per-customer-ID rate limits of about 1 QPS on the planning services. Scaling is horizontal across merchants and serial within a merchant.

Actions: apply for Basic immediately; complete Google Cloud brand verification (since July 2026 it can cut developer-token review to hours); submit the Standard application as soon as a demoable UI exists; and re-verify all quota figures at build time, since Google changes them.

19.2 Q2: Confirm whether PPCWay operates a single MCC or a hierarchy of sub-MCCs

Answer: a single MCC for Phases 1 and 2. Design for sub-MCC sharding but do not build it.

A manager account can link up to 85,000 non-manager accounts, which is far beyond Phase 1–3 volume. A single MCC is simpler to operate, simpler to reason about for `login-customer-id` handling, and avoids premature complexity.

Design for the future cheaply: every `google_ads_link` row stores the `manager_customer_id` it was linked under (§9.1), which turns a later migration into a data migration rather than a re-architecture. Reconsider sharding when any of these occurs: agency partners require their own manager-level isolation; a specific region needs a separate billing or data-residency boundary; or operational blast-radius concerns justify separating merchant cohorts.

One caveat: a brand-new MCC may be unable to create client accounts until it has accumulated spend history (§5.2), and the account-creation weekly ceiling depends on A1 Inc.’s payments profile type (§13.2). Both need confirmation with Google before Phase 2 is scheduled.

19.3 Q3: Confirm the LLM provider and the content-safety / policy-compliance validation approach

Answer: Anthropic’s Claude API, with Claude Opus 5 (`claude-opus-5`) as the default for every generative step. Policy compliance is validated deterministically, never by the model.

Model rationale in §7.2: ad copy quality is the product, generated text goes live under a merchant’s brand with money behind it, and LLM spend is roughly \$0.20–0.60 per merchant per month — about 3–5% of non-Stripe infrastructure cost (§7.7, §17). The cost case for a weaker model is therefore weak. Cost-down options are quantified in §7.7 so A1 Inc. can decide; the lowest-risk of them is moving the two narration tasks to Claude Haiku 4.5, which never touch Google.

Validation approach — the more important half of the answer. A three-layer defence, in which the model is never the judge of its own compliance:

1. **Constrain at generation.** Character limits, policy rules and prohibited claims are in the cached system prompt; output is forced through a JSON schema via structured outputs.
2. **Validate deterministically** (§7.4). Every check is code: character limits, prohibited superlatives and claims, trademark screening, capitalisation, punctuation, placeholder leakage, encoding, duplicate assets, and — most importantly — **landing-page substantiation**, requiring textual support on the crawled page for any claim, which is the single most effective defence against fabrication.
3. **Pre-flight against Google** with `validate_only = true` before anything is submitted for real, and before the merchant sees the preview.

Failure handling: two regeneration attempts with the violation fed back, then drop the asset if minimums still hold, then a human-written category template. Onboarding is never blocked by a model that will not produce compliant copy.

Also required: golden-set evaluation gating every prompt or model change (§18.3); prompt-version and model-ID recorded against every generated asset (§7.6); and treating crawled site content as untrusted input to the prompt, with the generation call holding no tools and no credentials (§12.6).

19.4 Q4: Confirm initial supported business verticals for launch

Answer: recommend launching on local services and professional practices, plus small-to-mid e-commerce; explicitly exclude Google’s restricted categories.

Vertical	Launch	Rationale
Local services (trades, home services, auto, salons, fitness)	Yes	Simplest campaign structure, clearest conversion (call/form), strongest fit for DRD’s “DIY Dana”
Professional practices (accounting, legal, consulting, dental non-medical)	Yes	High-value leads justify the subscription; Search-only keeps it explainable
Small/mid e-commerce	Yes	PMax fit; requires a working purchase conversion; feed optional
Restaurants / hospitality	Phase 2	Store visits and local inventory add complexity
B2B / SaaS	Phase 2	Long sales cycles make CPA optimization slow and the value story hard
Healthcare / medical	No	Restricted; certification required
Financial services / lending / crypto	No	Restricted; certification and local licensing
Gambling / alcohol / adult / political / pharma	No	Restricted or prohibited
Anything requiring local licensing verification	No	Verification burden PPCWay cannot discharge

Scoping consequence: the restricted-category gate (§6.1 and §12.3) is Phase 1 work, because it must block onboarding rather than fail at launch. It is a category-code decision table mapped to Google’s restricted-content policies, with an internal-admin override path for legitimate exceptions. Each additional vertical adds a category negative-keyword list, structure table rows, and prohibited-claim rules — roughly 2–4 engineer-days each, plus subject-matter expert time.

19.5 Q5: Confirm hosting/cloud provider preference and any existing A1 Inc. infrastructure to reuse

Answer: GCP is recommended, but weakly — if A1 Inc.’s existing “A2 stack” is on AWS, stay on AWS.

The genuine GCP advantages are adjacency, not capability: the Google Cloud project that owns the OAuth client and developer token is required regardless, brand verification lives there, and BigQuery is the natural Phase 3 analytics store given Google’s first-party Ads-to-BigQuery transfer. Nothing in this architecture is GCP-specific; §3.5 maps every service to its AWS equivalent.

Operational consistency with an existing stack, an existing on-call rotation and existing runbooks is worth more than that adjacency.

What should be reused from A1 Inc.’s existing infrastructure regardless of provider:

Reuse	Why
Transactional email provider and warmed sending domain	DRD §7 asks for this. Deliverability reputation takes time to build and critical alerts must not land in spam
Identity provider, if one is already in use	Avoids a second auth surface and a second set of security decisions
Observability and paging stack	One on-call surface; existing escalation policies
Terraform modules, CI patterns, base images	Meaningful Phase 0 saving
Stripe account, if A1 Inc. already has one in the right entity	Avoids re-verification and tax setup

What must be separate regardless: the Google Cloud project and Google Ads MCC (these are PPCWay-specific and carry the platform's most sensitive credentials), production database and secret stores, and Sentry/observability projects for clean signal separation.

Information still needed from A1 Inc. to close this question: the current provider and region of the A2 stack; whether an identity provider is already in place; the existing email provider and its sending domain; whether Terraform is already in use; and whether a corporate Stripe entity already exists.

A Appendix A — Glossary

Term	Meaning
Access level	The permission tier of a Google Ads API developer token: Test, Explorer, Basic or Standard. Determines the daily operation ceiling and which services are usable (§5.1)
Ad strength	Google's assessment of RSA/PMax asset quality and diversity. Influences auction eligibility
Asset group	The Performance Max container for text, image, logo and video assets
Autonomy level	PPCWay's per-merchant setting controlling whether automated changes apply automatically, require approval, or only raise alerts (§8.3)
Blast radius	The proportion of a merchant's entities a single automated job may affect. Capped by guardrail check 8 (§5.5)
Customer ID (CID)	A Google Ads account identifier. Several API rate limits are per-CID
Developer token	The platform-wide credential identifying PPCWay to the Google Ads API. One token serves all merchants
Envelope encryption	Encrypting data with a data key, then encrypting the data key with a KMS-managed key (§12.1)
GAQL	Google Ads Query Language, used for all reporting reads
Guardrail layer	The single component in <code>ads-gateway</code> that validates every proposed mutation against merchant limits before it may execute (§5.5)
Idempotency key	A deterministic identifier ensuring a retried mutation is applied at most once (§5.6)
Learning period	The interval after a bidding-strategy change during which Google's automation recalibrates. PPCWay suppresses further changes during it
<code>login-customer-id</code>	The request header identifying the manager account through which a client account is being accessed
MCC / Manager Account	A Google Ads manager account. PPCWay links merchant accounts beneath its own MCC (§5.2)
Operation	The unit of Google Ads API quota. One request may carry many operations; both reads and writes count (§5.4)
Optimization Action Log	The append-only record of every automated change, with before/after values (§8.4)
<code>partial_failure</code>	A Google Ads API mode where valid operations in a batch succeed even if others fail. The per-operation error list must be parsed (§5.6)
PMax / Performance Max	Google's cross-network automated campaign type
Prompt caching	Anthropic API feature that caches a stable prompt prefix, charged at roughly 0.1x on read (§7.3)
RMF	Required Minimum Functionality — the feature set Google requires of Standard Access tools (§12.3)
RSA	Responsive Search Ad. Up to 15 headlines and 4 descriptions, machine-assembled at auction
RLS	Row-level security. PostgreSQL enforcement of tenant isolation (§4.6)
Search term	The actual query a user typed, as distinct from the keyword that matched it
Structured outputs	Anthropic API feature forcing model output to conform to a JSON schema
Temporal activity	A single unit of I/O in a Temporal workflow, with its own retry policy
Temporal workflow	Durable, replayable orchestration code. Must be deterministic; all I/O lives in activities (§3.3)
<code>validate_only</code>	A Google Ads API request flag that validates a mutation without applying it (§5.6)

B Appendix B — Reference GAQL Queries

Illustrative queries for the reads described in §5.7 and §8.1. Field names must be checked against the pinned API version — see §5.8.

Campaign performance for the metrics store (daily ingest):

```
SELECT campaign.id, campaign.name, campaign.status,
       campaign.advertising_channel_type,
       campaign_budget.amount_micros,
       metrics.impressions, metrics.clicks, metrics.cost_micros,
       metrics.conversions, metrics.conversions_value,
       metrics.search_impression_share,
       metrics.search_budget_lost_impression_share,
       segments.date
FROM campaign
WHERE segments.date DURING LAST_7_DAYS
      AND campaign.status != 'REMOVED'
```

Search terms for the mining job:

```
SELECT search_term_view.search_term,
       search_term_view.status,
       campaign.id, ad_group.id,
       metrics.impressions, metrics.clicks, metrics.cost_micros,
       metrics.conversions
FROM search_term_view
WHERE segments.date DURING LAST_14_DAYS
      AND metrics.clicks > 0
```

Ad and asset policy status for the disapproval poll:

```
SELECT ad_group_ad.ad.id,
       ad_group_ad.policy_summary.approval_status,
       ad_group_ad.policy_summary.review_status,
       ad_group_ad.policy_summary.policy_topic_entries,
       campaign.id, ad_group.id
FROM ad_group_ad
WHERE ad_group_ad.policy_summary.approval_status IN
      ('DISAPPROVED', 'APPROVED_LIMITED', 'AREA_OF_INTEREST_ONLY')
```

Keyword-level performance for bid and pause decisions:

```
SELECT ad_group_criterion.criterion_id,
       ad_group_criterion.keyword.text,
       ad_group_criterion.keyword.match_type,
       ad_group_criterion.status,
       ad_group_criterion.approval_status,
       ad_group_criterion.quality_info.quality_score,
       metrics.impressions, metrics.clicks, metrics.cost_micros,
       metrics.conversions,
       segments.date
FROM keyword_view
WHERE segments.date DURING LAST_30_DAYS
      AND ad_group_criterion.status != 'REMOVED'
```

External-change detection (changes not made by PPCWay):

```
SELECT change_event.change_date_time,
       change_event.change_resource_type,
       change_event.resource_change_operation,
       change_event.changed_fields,
```

```

        change_event.client_type,
        change_event.user_email,
        change_event.campaign, change_event.ad_group
FROM change_event
WHERE change_event.change_date_time DURING LAST_2_DAYS
ORDER BY change_event.change_date_time DESC
LIMIT 1000

```

Note that `change_event` requires a `LIMIT` and is restricted to a trailing window (30 days at time of writing), and that `client_type` is what distinguishes a change made through the API by PPCWay from one made in the Google Ads UI by a human.

Account serviceability and billing state:

```

SELECT customer.id, customer.descriptive_name, customer.status,
       customer.currency_code, customer.time_zone,
       customer.pay_per_conversion_eligibility_failure_reasons
FROM customer

```

C Appendix C — Configuration Reference

Beyond the required set in §11.6, these tunables should be configuration rather than code, so that a Google policy change or a threshold adjustment does not require a deploy.

Key	Default	Purpose
GOOGLE_ADS_API_VERSION	pinned	Explicit API version (§5.8)
QUOTA_DAILY_CEILING	15000	Mirrors the token's tier; drives load shedding
QUOTA_ALERT_THRESHOLDS	60,80,92	Percent-of-ceiling alert points
QUOTA_SHED_LOW_PRIORITY_AT	80	Percent at which Low-priority jobs defer
CRAWL_MAX_PAGES	25	Politeness and cost bound
CRAWL_TIMEOUT_SECONDS	60	Total crawl budget
CRAWL_MAX_RPS_PER_HOST	1	Politeness
KEYWORD_MIN_MONTHLY_SEARCHES	scaled by budget	Deterministic keyword filter
KEYWORD_MAX_CPC_TO_BUDGET_RATIO	0.33	Excludes terms that exhaust the budget in <3 clicks
GUARDRAIL_MAX_CHANGE_PCT	25	Per-change delta cap
GUARDRAIL_MAX_DAILY_CHANGE_PCT	50	Cumulative daily delta cap
GUARDRAIL_MAX_PAUSE_PCT_PER_RUN	10	Blast-radius cap
GUARDRAIL_METRICS_STALENESS_SECONDS	21600	Six hours; fails closed beyond it
GUARDRAIL_MAX_CHANGES_PER_ENTITY_DAY	2	Velocity cap
APPROVAL_EXPIRY_DAYS	7	Approve-First proposal expiry
BIDDING_TCPA_MIN_CONVERSIONS	30	Threshold in the §6.5 decision table
BIDDING_LEARNING_PERIOD_DAYS	14	Change suppression window
MINING_NEGATE_MIN_CLICKS	12	Search-term negation threshold
MINING_PROMOTE_MIN_CONVERSIONS	2	Search-term promotion threshold
ANOMALY_SPEND_SPIKE_MULTIPLE	3.0	Versus trailing same-hour median
RSA_TARGET_HEADLINES	12	Generation target (max 15)
RSA_TARGET_DESCRIPTIONS	4	Generation target (max 4)
VALIDATOR_MAX_REGENERATION_ATTEMPTS	2	Before template fallback
POLICY_LIMITS_SOURCE	remote config	Character limits and asset minimums, so a Google change is a config update (§6.7)
RESTRICTED_CATEGORY_LIST	remote config	Blocked verticals (§6.1, §13.9)
LLM_MONTHLY_TOKEN_BUDGET	set per env	Cost circuit breaker
LLM_EFFORT_COPY	medium	<code>output_config.effort</code> for ad copy

Key	Default	Purpose
LLM_EFFORT_NARRATIVE	low	output_config.effort for narration

D Appendix D — Phase Acceptance Criteria

Each phase is complete when every criterion is demonstrably met, not when the code is written.

D.1 Phase 0

- ☐ MCC created under A1 Inc.'s corporate identity; developer token issued at Basic or above
- ☐ Google Cloud project created; brand verification submitted or complete
- ☐ OAuth client configured; consent screen submitted for verification
- ☐ A single authenticated Google Ads API call succeeds against the test account, from CI
- ☐ Three environments provisioned by Terraform from a clean state
- ☐ CI/CD deploys to staging automatically and to production behind manual approval
- ☐ Secrets resolve from Secret Manager via workload identity; no key files in images
- ☐ Observability dashboards and alert routing live; a test alert pages successfully

D.2 Phase 1 (MVP)

- ☐ A merchant can sign up, subscribe, link an existing Google Ads account, complete the wizard, review a plain-English preview, and launch a live Search campaign
- ☐ Median time-in-PPCWay for the above is under 15 minutes, excluding Google billing time
- ☐ Conversion tracking is verified by an observed test conversion before launch is permitted
- ☐ No mutation reaches Google except through ads-gateway; asserted by a CI static check
- ☐ Every mutation has an optimization_action row written before the API call
- ☐ Guardrails reject: over-cap budget, over-cap CPA, excessive delta, stale metrics, over-blast-radius, wrong-tenant entity — each covered by a passing test
- ☐ A worker killed mid-mutation leaves no duplicate change and reconciliation resolves it; demonstrated by the chaos test
- ☐ Approve-First proposals appear, can be approved and reverted, and expire after 7 days
- ☐ The policy validator rejects every case in its adversarial test set; no asset reaches Google without passing it
- ☐ **Advanced view exposes the RMF creation, management and reporting capabilities in §12.3**
- ☐ Cross-tenant isolation tests pass
- ☐ Data export and deletion flows work end to end
- ☐ Load test at 2x projected peak passes within latency objectives
- ☐ Standard Access application submitted with a demoable UI

D.3 Phase 2

- ☐ Standard Access granted
- ☐ Path B account creation works, with throughput queueing and clear merchant expectations
- ☐ Performance Max campaigns generate, validate and launch with compliant asset minimums
- ☐ Full-Auto mode is available only after separate recorded consent, and respects all guardrails
- ☐ Weekly digest sends via the Batch API, and correctly reports broken-state accounts rather than misleading performance numbers
- ☐ PDF/CSV export and custom date ranges available
- ☐ Bidding-strategy transitions occur automatically per §6.5 with learning-period suppression

D.4 Phase 3

- ☐ Agency console supports multi-client switching and consolidated reporting under a single login

- ☐ White-label theming and custom domains work for at least one pilot partner
 - ☐ Metrics store migrated to BigQuery with no dashboard regression
 - ☐ Sub-MCC sharding demonstrated by migrating a cohort without downtime
 - ☐ Microsoft Ads evaluation delivered as a written recommendation
-

END OF DOCUMENT

PPCWay — Technical Architecture & Implementation Blueprint, Version 1.0

Confidential — Internal & Vendor/Developer Use Only